

RapPhim WareHouse

Bộ tài liệu hướng dẫn

Tổng quan · Kiến trúc · Phát triển · API · Triển khai · Vận hành

Phiên bản 1.0 — 2026-09-11

Backend Spring Boot 4.1 (Java 17) · Frontend Next.js 16 (React 19)

Mục lục

Bộ tài liệu hướng dẫn RapPhim Warehouse	6
Giới thiệu	6
Bảng mục lục	6
Lộ trình đọc theo vai trò	7
Kết luận	7
Tổng quan hệ thống RapPhim Warehouse	9
Giới thiệu	9
Các thành phần chính	9
Kiến trúc hai tầng và luồng dữ liệu	10
Các nguồn phim và nguồn metadata	11
Chuẩn hoá về một schema	12
Cache Caffeine	12
Bề mặt API	13
Cấu hình và triển khai	13
Kết luận	14
Hướng dẫn cài đặt môi trường phát triển	15
Giới thiệu	15
Các thành phần chính	15
Yêu cầu phần mềm	15
Cài đặt và chạy backend	16
Cài đặt và chạy frontend	17
Chạy kiểm thử và kiểm tra	18
Kết luận	18
Kiến trúc hệ thống	19
Giới thiệu	19
Các thành phần chính	19
Sơ đồ các tầng và trách nhiệm	20
Trừu tượng Provider	20
Chuẩn hoá DTO	22
Vỏ bọc phản hồi và phân trang	23
Cache Caffeine	24

Cấu hình RestClient và timeout	26
CoreDNS với DNS-over-TLS	26
Luồng một request thực tế	27
Kết luận	28
Hướng dẫn thêm một nguồn phim (Provider) mới	30
Giới thiệu	30
Các thành phần chính	30
Trước khi bắt đầu	31
Bước 1 — Thêm giá trị vào enum ProviderType	31
Bước 2 — Khai báo cấu hình trong ProviderProperties và application.yml	32
Bước 3 — Tạo các record model Jackson	34
Bước 4 — Tạo RestClient bean trong RestClientConfig	36
Bước 5 — Cài đặt MovieProvider	37
Bước 6 — Thêm retry cho lỗi mạng chớp nhoáng	40
Bước 7 — Viết test cho provider mới	42
Kết luận	43
Tài liệu API	44
Giới thiệu	44
Các thành phần chính	44
Địa chỉ gốc và quy ước chung	44
Vỏ bọc phản hồi	45
Endpoint nhóm Movies	47
Endpoint nhóm Anime	47
Endpoint nhóm Homelab	48
Endpoint nhóm Taxonomies	48
Endpoint nhóm TMDB & NFO	48
Endpoint nhóm Quản trị	49
Endpoint nhóm System	50
Bảng mã lỗi	50
Ví dụ request và response	51
Swagger UI và OpenAPI JSON	53
Kết luận	54
Hướng dẫn triển khai bằng Docker (homelab)	55

Giới thiệu	55
Các thành phần chính	55
Chuẩn bị tệp .env	56
Build và chạy tại chỗ	57
Triển khai từ xa bằng deploy.sh	58
CoreDNS: bộ phân giải tên miền riêng	59
Health-check, xem log, cập nhật, rollback	60
Lưu ý tài nguyên: dịch vụ thuyết minh (dub) đang tắt	61
Kết luận	61
Tham chiếu cấu hình	62
Giới thiệu	62
Hai lớp biến môi trường	62
Các lớp cấu hình	63
Cổng và URL	63
Nguồn ZCloud (kho phim homelab)	64
TheMovieDB (TMDB)	65
Nguồn metadata anime (AniList và Jikan)	66
Khoá quản trị (Admin token)	66
Nguồn phim: base URL và timeout	67
CORS	68
Giới hạn RAM (tùy chọn)	69
Tổng hợp hệ quả khi để trống	69
Kết luận	69
Hướng dẫn xử lý sự cố	71
Giới thiệu	71
Kỷ luật chẩn đoán phân tầng	71
Bảng tra nhanh: triệu chứng nguyên nhân cách xử lý	72
Kiểm tra sức khoẻ trước khi chẩn đoán	73
Sự cố 1 — 502 UPSTREAM_ERROR ở VSMOV (bài học phân tầng)	74
Sự cố 2 — nguồn homelab trả rỗng	75
Sự cố 3 — TMDB trả 503 TMDB_NOT_CONFIGURED	76
Sự cố 4 — trình duyệt chặn API do CORS	77
Sự cố 5 — ISP đầu độc DNS, và chặn TLS theo tên miền	77
Sự cố 6 — homelab hết RAM, OOM-killer giết dịch vụ	79

Kết luận	79
--------------------	----

Bộ tài liệu hướng dẫn RapPhim WareHouse

Tài liệu: Mục lục bộ hướng dẫn RapPhim WareHouse

Phiên bản: 1.0

Cập nhật: 2026-09-11

Đối tượng đọc: Mọi người tham gia dự án (lập trình viên, người tích hợp API, DevOps)

Trạng thái: Chính thức

Tag: mục-lục, hướng-dẫn, tổng-quan, backend, frontend, devops

Giới thiệu

Đây là mục lục của bộ tài liệu hướng dẫn RapPhim WareHouse — một kho phim web gồm **backend Spring Boot 4.1.1** (Java 17, Maven Wrapper) đóng vai trò lớp trung gian chuẩn hoá nhiều nguồn phim về một schema DTO duy nhất, và **frontend Next.js 16.3.4** (React 19.2.8, Tailwind CSS 4) hiển thị giao diện xem phim.

Bộ tài liệu được viết để đọc theo nhu cầu: mỗi file đứng độc lập với một khối metadata và một kết luận trở tới các tài liệu liên quan, nhưng cả bộ cũng ghép thành một mạch từ tổng quan, cài đặt, kiến trúc, mở rộng nguồn, API, triển khai, cấu hình đến xử lý sự cố. Trang này cho bạn hai thứ: một **bảng mục lục** để nhảy thẳng tới tài liệu cần đọc, và một **lộ trình đọc theo vai trò** để biết nên đọc theo thứ tự nào.

Toàn bộ nội dung được kiểm chứng trực tiếp từ mã nguồn trong `backend/src/main/java/com/raphim/warehouse/` và các tệp hạ tầng ở gốc kho, nên khớp với hiện thực. Khi cần tra cứu chính xác từng endpoint, hãy mở Swagger UI hoặc bản xuất tĩnh `docs/openapi.json`.

Bảng mục lục

STT	Tài liệu	Mô tả	Đối tượng
00	Tổng quan hệ thống	Bức tranh toàn cảnh: các thành phần, luồng dữ liệu, danh sách nguồn phim / metadata, cache và bề mặt API.	Người mới tiếp cận dự án, quản lý kỹ thuật, người tích hợp
01	Cài đặt môi trường phát triển	Dựng backend và frontend tại máy, biến môi trường tối thiểu, chạy kiểm thử và kiểm tra.	Lập trình viên mới tham gia dự án
02	Kiến trúc hệ thống	Phân tầng web / service / provider / config, trừu tượng <code>MovieProvider</code> , chuẩn hoá DTO, cache Caffeine, CoreDNS và luồng một request thật.	Lập trình viên backend, kiến trúc sư
03	Thêm một nguồn phim (Provider) mới	Bảy bước thêm nguồn ở mức mã nguồn: enum, cấu hình, model Jackson, <code>RestClient</code> , cài đặt <code>MovieProvider</code> , retry và test.	Lập trình viên backend mở rộng nguồn phim
04	Tài liệu API	Quy ước chung, vỏ bọc phản hồi, danh sách endpoint theo nhóm, bảng mã lỗi và ví dụ gọi thật.	Người tích hợp API, lập trình viên frontend
05	Triển khai bằng Docker (homelab)	Ba container <code>dns</code> / <code>backend</code> / <code>web</code> , chuẩn bị <code>.env</code> , build và chạy, <code>deploy.sh</code> , CoreDNS, health-check và rollback.	DevOps / người vận hành homelab

STT	Tài liệu	Mô tả	Đối tượng
06	Tham chiếu cấu hình	Bản tham chiếu đầy đủ biến môi trường, ánh xạ sang lớp <i>*Properties</i> , và hệ quả khi để trống từng biến.	DevOps, lập trình viên
07	Xử lý sự cố	Kỷ luật chẩn đoán phân tầng, bảng tra nhanh triệu chứng, các sự cố lớn kèm lệnh kiểm tra, và checklist chẩn đoán.	DevOps, trực vận hành, lập trình viên

Lộ trình đọc theo vai trò

Không cần đọc cả bộ theo số thứ tự. Chọn lộ trình khớp vai trò của bạn.

Lập trình viên mới

Mục tiêu: hiểu hệ thống rồi bắt đầu viết mã được.

- 00 – Tổng quan hệ thống — nắm hệ thống gồm những gì và dữ liệu chảy ra sao.
- 01 – Cài đặt môi trường phát triển — dựng máy, chạy được backend và frontend.
- 02 – Kiến trúc hệ thống — hiểu cách phân tầng và trừu tượng provider.
- 04 – Tài liệu API — bề mặt API mà frontend và tầng service làm việc cùng.
- 03 – Thêm một nguồn phim mới — bài thực hành mở rộng đầu tiên khi đã quen mã.
- 07 – Xử lý sự cố — kỷ luật "phân tầng trước, quy kết sau" để gỡ lỗi.

Người tích hợp API

Mục tiêu: gọi API đúng và xử lý lỗi gọn.

- 00 – Tổng quan hệ thống — hiểu vai trò lớp trung gian và tham số *provider*.
- 04 – Tài liệu API — endpoint theo nhóm, vỏ bọc *ApiResponse* / *PageResponse*, bảng mã lỗi.
- 06 – Tham chiếu cấu hình — phần CORS và các khoá dịch vụ ảnh hưởng tới lời gọi.
- 07 – Xử lý sự cố — đọc trường *code* để rẽ nhánh xử lý khi API trả lỗi.

DevOps

Mục tiêu: triển khai, cấu hình và vận hành ổn định.

- 00 – Tổng quan hệ thống — kiến trúc và ánh xạ cổng (nội bộ 8080/3000, homelab 7101/7100).
- 05 – Triển khai bằng Docker (homelab) — dựng ba container, *deploy.sh*, CoreDNS, cập nhật và rollback.
- 06 – Tham chiếu cấu hình — biến *.env*, biến container, và hệ quả khi để trống.
- 07 – Xử lý sự cố — chẩn đoán DNS/TLS, CORS, OOM và các sự cố vận hành.

Kết luận

Bộ tài liệu này bao trọn vòng đời làm việc với RapPhim WareHouse: từ hiểu hệ thống, dựng môi trường, nắm kiến trúc, mở rộng nguồn phim, tích hợp API, triển khai homelab, tra cứu cấu hình đến xử lý sự cố. Hãy dùng bảng mục lục để nhảy tới đúng tài liệu, và lộ trình theo vai trò để đọc theo thứ tự hợp lý.

Tài liệu và tài nguyên liên quan ngoài bộ hướng dẫn:

- ../openapi.json* và Swagger UI (<http://localhost:8080/swagger-ui.html>) — đặc tả API đầy đủ, gọi thử được.

- `../ebook/README.md` — sổ tay sự cố DevOps thật rút từ nhật ký triển khai dự án.
- `../../README.md` — sổ tay dự án ở gốc kho: tổng quan, cách chạy và các giới hạn đã biết.
- `../../AGENTS.md` — quy ước vận hành, ranh giới an toàn và bản đồ dự án cho agent.

Tổng quan hệ thống RapPhim WareHouse

Tài liệu: Tổng quan hệ thống RapPhim WareHouse

Phiên bản: 1.0

Cập nhật: 2026-09-11

Đối tượng đọc: Người mới tiếp cận dự án, quản lý kỹ thuật, người tích hợp

Trạng thái: Chính thức

Tag: tổng-quan, kiến-trúc, backend, frontend, nguồn-phim, metadata

Giới thiệu

RapPhim WareHouse là một kho phim web. Chúng ta xây nó để giải quyết một vấn đề rất cụ thể: có nhiều nguồn phim khác nhau, mỗi nguồn trả về JSON theo một định dạng riêng, và không nguồn nào giống nguồn nào. Nếu để frontend tự gọi từng nguồn, giao diện sẽ phải hiểu bốn "phương ngữ" JSON, xử lý bốn kiểu phân trang, bốn cách đặt tên trường — mỗi lần thêm nguồn là một lần sửa lại toàn bộ giao diện.

Hệ thống đặt một **lớp trung gian** ở giữa để gánh việc đó. Backend gọi các nguồn bên ngoài, đưa những định dạng khác nhau về **một schema DTO duy nhất**, cache lại kết quả rồi phục vụ frontend. Nhờ vậy frontend chỉ cần biết một hợp đồng dữ liệu, không cần biết nguồn nào đứng sau. Thêm nguồn mới nghĩa là viết thêm một provider ở backend, còn giao diện gần như không phải đổi.

Ngoài phim, backend còn tổng hợp **metadata**: thông tin phim điện ảnh từ TMDB (kèm xuất file NFO cho trình quản lý media) và metadata anime từ AniList / Jikan. Toàn bộ API được mô tả bằng Swagger UI sinh tự động từ annotation trong mã, nên tài liệu không bao giờ lệch với hiện thực.

Tài liệu này cho bạn bức tranh toàn cảnh: hệ thống gồm những gì, dữ liệu chảy ra sao, và đọc tiếp ở đâu.

Các thành phần chính

Dự án gồm hai tầng chạy độc lập, cộng một số dịch vụ hạ tầng khi triển khai bằng Docker.

Thành phần	Công nghệ	Cổng	Vai trò
backend/	Spring Boot 4.1.1 (Java 17, Maven Wrapper)	8080	REST API tổng hợp nhiều nguồn phim, chuẩn hoá về một schema DTO, bổ sung metadata TMDB / AniList / Jikan, cache Caffeine, có Swagger UI
frontend /	Next.js 16.3.4 (App Router), React 19.2.8, Tailwind CSS 4, hls.js	3000	Giao diện xem phim theo phong cách YouTube; mọi lời gọi API đi qua route handler cùng gốc của chính Next
dns (khi deploy)	CoreDNS 1.14.7 (DNS-over-TLS)	nội bộ	Bộ phân giải tên miền riêng để vượt chặn DNS của ISP; container raphim-dns ở IP tĩnh 172.29.0.53

Trên homelab, cổng 8080 đã có dịch vụ khác (ZCloud) dùng rồi, nên backend được **đưa ra ngoài ở cổng 7101** và web ở **7100**. Bên trong mạng Docker, hai container vẫn nghe 8080 và 3000 như bình thường.

Container	Ảnh xạ cổng (homelab)	Cổng nội bộ
rapphim-backend	7101	8080
rapphim-web	7100	3000
rapphim-dns	— (chỉ nội bộ)	53

Kiến trúc hai tầng và luồng dữ liệu

Frontend không bao giờ gọi thẳng backend từ trình duyệt. Mọi lời gọi đi qua route handler cùng gốc của Next (/api/*), route handler này mới gọi backend. Backend đứng trước một loạt nguồn bên ngoài, chuẩn hoá dữ liệu và cache lại.

mermaid

flowchart LR

```

    B[Trình duyệt] -->|HTTP cung gốc| N[Next.js\nroute handler /api/*]
    N -->|HTTP noi bo| S[Spring Boot REST API\n/api/v1]
    S --> C{Caffeine cache}
    S -->|nguồn phim| K[KKPhim\nphimapi.com]
    S -->|nguồn phim| G[NguồnC\nphim.nguonc.com]
    S -->|nguồn phim| V[VSMOV\nvsmov.com]
    S -->|nguồn phim| Z[Homelab / ZCloud\nFastAPI + S3]
    S -->|metadata phim| T[TMDB\napi.themoviedb.org]
    S -->|metadata anime| A[AniList\ngraphql.anilist.co]
    S -->|metadata anime| J[Jikan / MyAnimeList\napi.jikan.moe]
  
```

Nếu môi trường của bạn không dựng được sơ đồ Mermaid, đây là cùng một luồng dưới dạng văn bản:

Trình duyệt

```

    | HTTP cung gốc (route handler /api/* của Next)
    v
  
```

Next.js (frontend, cổng 3000 / homelab 7100)

```

    | HTTP noi bo -> API_INTERNAL_URL
    v
  
```

Spring Boot REST API (backend, cổng 8080 / homelab 7101)

```

    |
    +--> Caffeine cache (tra ngay neu con han)
    |
  
```

```

    +--> KKPhim      (phimapi.com)      -- nguồn phim
    +--> NguồnC      (phim.nguonc.com)  -- nguồn phim
    +--> VSMOV       (vsmov.com)       -- nguồn phim
    +--> Homelab     (ZCloud: FastAPI + S3) -- kho phim riêng
    +--> TMDB        (api.themoviedb.org/3) -- metadata phim + NFO
    +--> AniList     (graphql.anilist.co) -- metadata anime
    +--> Jikan/MAL   (api.jikan.moe/v4)  -- metadata anime (du phong)
  
```

Backend nhận rất nhiều định dạng JSON đầu vào và trả ra **một** envelope thống nhất cho frontend:

java

```
// common/ApiResponse.java - envelope chuan cho moi response
public record ApiResponse<T>{
    boolean success,    // true neu request duoc xu ly thanh cong
    String message,    // thong diep mo ta ket qua
    T data,            // du lieu nghiep vu
    Instant timestamp // thoi diem server tra ve (UTC)
} { }
```

Các nguồn phim và nguồn metadata

Danh sách nguồn phim là một enum ở backend; mỗi nguồn có một mã ngắn (code) truyền qua tham số `provider` của các endpoint.

java

```
// dto/ProviderType.java - cac nguon du lieu phim ma he thong ho tro
public enum ProviderType {
    KKPHIM("kkphim"),    // KKPhim - phimapi.com
    NGUONC("nguonc"),    // NguonC - phim.nguonc.com
    VSMOV("vsmov"),      // VSMOV - vsmov.com
    HOMELAB("homelab"); // Kho phim riêng tren homelab, doc qua ZCloud
}
```

Các nguồn cấu hình trong backend/src/main/resources/application.yml:

Nguồn	Mã	Loại	Địa chỉ	Ghi chú
KKPhim	kkphim	Nguồn phim	https://phimapi.com	Nguồn phim mặc định khi không truyền provider
NguonC	nguonc	Nguồn phim	https://phim.nguonc.com	Định dạng JSON khác hẳn KKPhim, được chuẩn hoá về cùng schema
VSMOV	vsmov	Nguồn phim	https://vsmov.com	Ảnh là URL tuyệt đối nên không cần gốc CDN
Homelab	homelab	Kho phim riêng	ZCloud (FastAPI + S3), mặc định http://192.168.100.169:8080	Đọc qua ZCloud bằng <code>ZCLOUD_API_KEY</code> hoặc <code>ZCLOUD_PASSWORD</code> ; thiếu cả hai thì nguồn này trả rỗng, phần còn lại vẫn chạy
TMDB	—	Metadata phim	https://api.themoviedb.org/3	Làm giàu chi tiết phim và xuất file NFO; cần <code>TMDB_ACCESS_TOKEN</code> (token v4) hoặc <code>TMDB_API_KEY</code> (khóa v3), thiếu cả hai thì các endpoint TMDB trả 503
AniList	—	Metadata anime	https://graphql.anilist.co	GraphQL, không cần khoá, không phát video

Nguồn	Mã	Loại	Địa chỉ	Ghi chú
Jikan / My AnimeList	—	Metadata anime	https://api.jikan.moe/v4	REST, không khoá; nguồn dự phòng cho AniList

Nguồn phim và nguồn metadata anime tuân theo hai hợp đồng interface riêng (`MovieProvider` và `AnimeMetadataProvider`), nên thêm một nguồn thứ n chỉ cần hiện thực đúng interface tương ứng — phần còn lại của hệ thống không phải biết.

Chuẩn hoá về một schema

Đây là triết lý gốc của backend: bốn nguồn phim với bốn định dạng JSON được ép về **một** DTO chung. Frontend chỉ làm việc với DTO này, không bao giờ chạm định dạng thô của nguồn.

```
java
// dto/MovieSummary.java - schema tóm tắt một phim (đã chuẩn hoá)
public record MovieSummary(
    String id, String slug, String name, String originName,
    String posterUrl, String thumbUrl, Integer year, String type,
    String quality, String lang, String time, String episodeCurrent,
    List<Taxonomy> categories, List<Taxonomy> countries,
    TmdbRef tmdb, ImdbRef imdb, // định danh TMDb / IMDb, null nếu nguồn không cung cấp
    String provider, // nguồn đã trả về phim này
    String modifiedAt
) { }
```

Trường `provider` đi kèm mỗi phim cho biết nó đến từ nguồn nào — dữ liệu đã thống nhất về hình dạng, nhưng vẫn truy được nguồn gốc.

Cache Caffeine

Gọi lại nguồn ngoài cho mỗi request là lãng phí và chậm. Backend đặt một lớp cache trong bộ nhớ (Caffeine) trước các nguồn, mỗi loại dữ liệu có tuổi thọ riêng theo mức độ thay đổi của nó. Mọi cache đều có `maximumSize` (quy ước bắt buộc của repo — cache không chặn kích thước là rò rỉ bộ nhớ chờ sẵn).

Cache		Hết hạn sau	Lý do
<code>movieLists</code>	500	5 phút	Danh sách phim đổi mới nhanh nên giữ ngắn
<code>movieDetails</code>	1000	30 phút	Chi tiết phim ổn định hơn, giữ lâu hơn
<code>taxonomies</code>	32	12 giờ	Thể loại / quốc gia gần như tĩnh
<code>homeLibrary</code>	4	3 phút	Cây file kho riêng; giữ ngắn để phim mới copy vào là thấy ngay, không phải khởi động lại

Cấu hình đầy đủ ở `backend/src/main/java/com/rapphim/warehouse/config/CacheConfig.java`.

Bề mặt API

Toàn bộ endpoint nằm dưới tiền tố `/api/v1`. Bảng dưới liệt kê các nhóm chính theo controller để bạn định vị nhanh; danh sách đầy đủ và có thể gọi thử nằm ở Swagger UI.

Nhóm	Tiền tố	Vai trò
Phim	<code>/api/v1/movies</code>	Mới nhất, theo loại, tìm kiếm, theo thể loại / quốc gia / năm, chi tiết theo slug, lấy nhiều phim (<code>/batch</code>)
Anime	<code>/api/v1/anime</code>	Thịnh hành, tìm kiếm, chi tiết theo id, danh sách thể loại
Homelab	<code>/api/v1/homelab</code>	Phục vụ tài nguyên kho riêng, ví dụ phụ đề (<code>/subtitle</code> , trả <code>text/vtt</code>)
TMDB / NFO	<code>/api/v1</code>	Metadata TMDB cho một phim, xuất NFO (<code>/movies/{slug}/nfo</code>), khám phá, chi tiết, nhân vật, thể loại
Hệ thống	<code>/api/v1</code>	<code>/status</code> (tình trạng cấu hình), <code>/providers</code> (mã nguồn đang hoạt động), <code>/list-types</code>
Taxonomy	<code>/api/v1</code>	<code>/categories</code> , <code>/countries</code>
Quản trị	<code>/api/v1/admin</code>	Nhật ký kiểm toán, cấu hình khoá, quản lý nguồn tự thêm (cần <code>RAPPHIM_ADMIN_TOKEN</code>)

Địa chỉ tài liệu và giám sát:

Đường dẫn	Nội dung
<code>http://localhost:8080/swagger-ui.html</code>	Swagger UI, gọi thử trực tiếp
<code>http://localhost:8080/v3/api-docs</code>	OpenAPI 3.1 JSON (sinh tự động)
<code>docs/openapi.json</code>	Bản xuất tĩnh của đặc tả OpenAPI
<code>http://localhost:8080/actuator/health</code>	Health check

Thử nhanh health check của backend:

```
bash
# Kiểm tra backend con song (chạy khi backend đang bật)
curl -s http://localhost:8080/actuator/health
```

Kết quả:

```
json
{"status": "UP"}
```

Cấu hình và triển khai

Backend đọc các khoá nhạy cảm từ biến môi trường (hoặc trang quản trị */cms*), không bao giờ hardcode. Các biến thường gặp:

Biến môi trường	Vai trò	Thiếu thì sao
RAPPHIM_CORS_ORIGINS	Địa chỉ frontend được phép gọi API	Trình duyệt chặn lời gọi khác gốc
TMDB_ACCESS_TOKEN / TMDB_API_KEY	Khoá TMDB (token v4 ưu tiên)	Các endpoint TMDB trả 503, phần còn lại chạy
ZCLOUD_API_KEY / ZCLOUD_PASSWORD	Truy cập kho riêng homelab	Nguồn homelab tự tắt, phần còn lại chạy
RAPPHIM_ADMIN_TOKEN	Bảo vệ trang quản trị	Chỉ xem được, không sửa được nguồn
API_INTERNAL_URL (web)	Địa chỉ backend mà route handler của Next gọi tới	Frontend không gọi được backend

Triển khai homelab qua `deploy.sh` (đóng gói `tar` rồi gửi qua SSH) kết hợp `docker compose` với ba dịch vụ `dns` / `backend` / `web`. Bản phát hành dùng `docker-compose.prod.yml` kéo ảnh dựng sẵn từ GHCR chứ không build trên VPS. Dịch vụ `dns` chạy CoreDNS với DNS-over-TLS để vượt chặn DNS của ISP; container backend được trỏ về bộ phân giải này bằng địa chỉ IP tĩnh `172.29.0.53` (ở mức đó Docker chưa nhận tên dịch vụ).

Dịch vụ thuyết minh (dub) **hiện đang tắt** trong `docker-compose.yml`: khối ML nặng (PaddleOCR ONNX + ffmpeg + TTS) quá sức homelab hiện tại. Mã nguồn vẫn còn ở thư mục `dub/` và tầng giao diện; muốn bật lại thì thêm service, đặt `DUB_INTERNAL_URL` cho web và bật cờ `NEXT_PUBLIC_DUB_ENABLED`.

Kết luận

RapPhim WareHouse gom nhiều nguồn phim và nhiều nguồn metadata về sau **một** lớp backend Spring Boot: backend chuẩn hoá tất cả về một schema DTO, cache bằng Caffeine theo tuổi thọ từng loại dữ liệu, rồi phục vụ một frontend Next.js gọi qua route handler cùng gốc. Kiến trúc này giữ cho giao diện đơn giản và cho phép thêm nguồn mới mà không đụng tới phần còn lại.

Đọc tiếp:

- `README.md` — cách chạy backend và frontend tại máy.
- `AGENTS.md` — hiến pháp cho agent: quy ước vận hành, ranh giới an

toàn, lệnh chuẩn và bản đồ dự án.

- `docs/openapi.json` và Swagger UI — đặc tả API đầy đủ, gọi thử được.
- `docs/ebook/README.md` — sổ tay sự cố thật và các nguyên tắc

vận hành rút ra từ dự án.

Hướng dẫn cài đặt môi trường phát triển

Tài liệu

Phiên bản: 1.0

Cập nhật: 2026-09-11

Đối tượng đọc: Lập trình viên mới tham gia dự án

Trạng thái: Chính thức

Tag: cài đặt, môi trường phát triển, backend, frontend, Spring Boot, Next.js

Giới thiệu

RapPhim Warehouse gồm hai phần chạy độc lập: một **backend Spring Boot** đóng vai trò lớp trung gian gọi nhiều nguồn phim rồi chuẩn hoá về một schema DTO duy nhất, và một **frontend Next.js** hiển thị giao diện xem phim. Trước khi viết được dòng mã đầu tiên, chúng ta cần dựng đúng môi trường để cả hai phần khởi động, nói chuyện được với nhau và vượt qua bộ kiểm thử.

Tài liệu này hướng dẫn từng bước từ lúc máy còn trống tới lúc mở được trang chủ tại <http://localhost:3000> và gọi được API tại <http://localhost:8080>. Chúng ta cố tình giữ bước cài đặt gọn nhất có thể: dự án dùng **Maven Wrapper** nên không phải cài Maven, và mọi khoá dịch vụ ngoài đều **tuỳ chọn** — thiếu khoá thì phần liên quan tự xuống thang chứ không chặn cả hệ thống. Nhờ vậy một lập trình viên mới có thể chạy được toàn bộ luồng chính chỉ với JDK và Node.js.

Các thành phần chính

Thành phần	Công nghệ	Cổng cục bộ	Vai trò
backend/	Spring Boot 4.1.1 (Java 17), Maven Wrapper	8080	REST API tổng hợp và chuẩn hoá dữ liệu từ các nguồn phim, cache Caffeine, sinh file NFO, có Swagger UI
frontend/	Next.js 16.3.4, React 19.2.8, Tailwind CSS 4	3000	Giao diện xem phim theo phong cách YouTube

Backend gọi ra các nguồn bên ngoài (KKPhim, NguonC, VSMOV, kho riêng homelab ZCloud), bổ sung metadata từ TMDB và AniList/Jikan, rồi phục vụ frontend qua một schema duy nhất. Frontend không cần biết nguồn nào trả về gì.

Yêu cầu phần mềm

Chúng ta cần đúng hai công cụ trên máy phát triển:

Công cụ	Phiên bản tối thiểu	Ghi chú
JDK	17 trở lên	Backend khai báo <code>java.version = 17</code> trong <code>backend/pom.xml</code>
Node.js	20 trở lên	Kèm sẵn <code>npm</code> ; frontend dùng <code>@types/node</code> bản 20

Không cần cài Maven — dự án đi kèm Maven Wrapper (`mvnw` / `mvnw.cmd`) tự tải đúng phiên bản Maven khi chạy lần đầu.

Kiểm tra nhanh phiên bản đã cài:

```
bash
java -version
node -v
```

Cài đặt và chạy backend

Backend nằm trong thư mục `backend/`. Từ gốc kho, chuyển vào thư mục đó rồi gọi Maven Wrapper với goal `spring-boot:run`. Lần chạy đầu sẽ tải phụ thuộc nên hơi lâu.

```
bash
cd backend && ./mvnw spring-boot:run
```

Trên Windows (PowerShell hoặc CMD), dùng bản `.cmd` của wrapper:

```
bat
cd backend
mvnw.cmd spring-boot:run
```

Khi ứng dụng khởi động xong, backend nghe ở cổng `8080` (khai báo trong `backend/src/main/resources/application.yml`). Các địa chỉ chính:

Mục	URL
Gốc REST API	< http://localhost:8080/api/v1 >
Swagger UI	< http://localhost:8080/swagger-ui.html >
OpenAPI JSON	< http://localhost:8080/v3/api-docs >
Health check	< http://localhost:8080/actuator/health >

Kiểm tra ứng dụng đã sống bằng health-check của Spring Boot Actuator:

```
bash
curl http://localhost:8080/actuator/health
```

Kết quả: khi ứng dụng khỏe, endpoint trả về trạng thái `UP`.

```
json
{ "status": "UP" }
```

Cấu hình `management.endpoint.health.show-details: always` nên phản hồi thật còn kèm một khối `components` mô tả chi tiết từng thành phần. Trường quan trọng nhất để biết ứng dụng đã sẵn sàng là `status`.

Gọi thử một endpoint nghiệp vụ để chắc chắn API trả dữ liệu thật:


```
bash
```

```
curl "http://localhost:8080/api/v1/movies/latest?page=1&limit=12"
```

Biến môi trường tối thiểu

Để chạy được luồng chính, backend **không bắt buộc biến môi trường nào** — mọi khoá trong `application.yml` đều có giá trị mặc định và tự xuống thang khi để trống. Các biến dưới đây là **tùy chọn**, chỉ đặt khi cần bật thêm tính năng:

Biến	Tác dụng khi đặt	Khi để trống
TMDB_ACCESS_TOKEN	Token đọc v4 của TheMovieDB (ưu tiên)	Nhóm endpoint <code>/api/v1/tmdb/*</code> trả 503 TMDB_NOT_CONFIGURED; xuất NFO vẫn chạy
TMDB_API_KEY	Khoá API v3 của TheMovieDB	Như trên
ZCLOUD_API_KEY	Khoá gọi kho phim riêng homelab (header <code>x-api-key</code>)	Nguồn homelab trả về rỗng
ZCLOUD_PASSWORD	Mật khẩu đăng nhập kho riêng, dùng khi chưa tạo khoá	Nguồn homelab trả về rỗng
RAPPIM_ADMIN_TOKEN	Khoá bật phần sửa đổi của trang quản trị	Trang quản trị chỉ còn xem được
RAPPIM_CORS_ORIGINS	Danh sách origin được phép gọi API	Mặc định <code>http://localhost:3000,http://127.0.0.1:3000</code>

Đặt biến trước khi chạy, ví dụ với TMDB:

```
bash
```

```
export TMDB_ACCESS_TOKEN="<read access token v4>"
cd backend && ./mvnw spring-boot:run
```

Danh sách đầy đủ các biến, ý nghĩa và cách cấu hình cho từng môi trường nằm ở tài liệu o6 – Tham chiếu cấu hình.

Cài đặt và chạy frontend

Frontend nằm trong thư mục `frontend/`. Cài phụ thuộc bằng `npm install`, sau đó chạy máy chủ phát triển bằng `npm run dev`.

```
bash
```

```
cd frontend && npm install && npm run dev
```

Mở `<http://localhost:3000>` để xem giao diện. Frontend đọc địa chỉ backend từ biến `NEXT_PUBLIC_API_BASE_URL`; giá trị mặc định trở về backend cục bộ. Chép file mẫu rồi sửa nếu cần:

```
bash
```

```
cp .env.example .env.local
```

Nội dung mặc định của `frontend/.env.local`:

```
bash
```

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8080
```

Với địa chỉ mặc định này, frontend ở cổng 3000 sẽ gọi backend ở cổng 8080 mà không phải chỉnh gì thêm.

Chạy kiểm thử và kiểm tra

Trước khi mở pull request, chúng ta chạy bộ kiểm thử của backend và bước kiểm tra type/lint của frontend.

Kiểm thử backend bằng Maven Wrapper (bao gồm test provider, sinh NFO, tham số discover và MockMvc):

```
bash
```

```
cd backend && ./mvnw test
```

Kiểm tra frontend gồm hai bước — build production chạy luôn kiểm tra kiểu TypeScript, và ESLint soát lỗi phong cách:

```
bash
```

```
cd frontend && npm run build  
cd frontend && npx eslint .
```

Cả hai bước cần chạy sạch trước khi gửi thay đổi. Nếu backend đang bật, frontend sẽ gọi được dữ liệu thật khi build; nếu không, các trang vẫn build được và chỉ hiển thị dữ liệu khi backend sẵn sàng.

Kết luận

Đến đây chúng ta đã có một môi trường phát triển đầy đủ: backend Spring Boot chạy ở cổng 8080 với Swagger UI và health-check, frontend Next.js chạy ở cổng 3000 trở về backend qua NEXT_PUBLIC_API_BASE_URL, cùng bộ kiểm thử và kiểm tra để bảo đảm thay đổi không làm hỏng luồng hiện có. Điểm mấu chốt cần nhớ: dự án dùng Maven Wrapper nên không cần cài Maven, và mọi khoá dịch vụ ngoài đều tùy chọn — luồng chính chạy được ngay chỉ với JDK 17+ và Node.js 20+.

Các tài liệu liên quan trong bộ hướng dẫn:

- 06 – Tham chiếu cấu hình — danh sách đầy đủ các

biến môi trường và cách cấu hình cho từng môi trường.

- Tài liệu API sinh tự động: mở Swagger UI tại <http://localhost:8080/swagger-ui.html> hoặc

xem bản xuất tinh ở docs/openapi.json.

Kiến trúc hệ thống

Tài liệu: Kiến trúc hệ thống RapPhim Warehouse
 Phiên bản: 1.0
 Cập nhật: 2026-09-11
 Đối tượng đọc: Lập trình viên backend, kiến trúc sư
 Trạng thái: Chính thức
 Tag: kien-truc, backend, provider, cache, dns

Giới thiệu

RapPhim Warehouse là một kho phim web gồm hai phần chạy độc lập: backend Spring Boot 4.1 (Java 17, dùng Maven Wrapper) nghe cổng 8080, và frontend Next.js 16 + React 19 + Tailwind 4 nghe cổng 3000. Trên homelab hai tầng được đưa ra ngoài ở cổng 7101 (backend) và 7100 (web).

Điều cốt lõi cần nắm trước khi đọc mã: **backend không phải là chủ của dữ liệu, nó là một lớp trung gian**. Mỗi bộ phim, mỗi tập, mỗi thể loại đều đến từ một nguồn bên ngoài (KKPhim, NguonC, VSMOV, kho riêng homelab), còn metadata thì đến từ TMDB, AniList và Jikan. Các nguồn này trả về JSON theo những định dạng rất khác nhau. Nhiệm vụ của backend là gọi các nguồn đó, đưa mọi thứ về **một schema DTO duy nhất**, đệm (cache) lại kết quả rồi phục vụ frontend. Nhờ vậy frontend không cần biết nguồn nào trả về gì, và thêm một nguồn mới không làm thay đổi tầng web.

Tài liệu này giải thích cách các tầng được xếp đặt, tầng nào chịu trách nhiệm gì, và một request thật đi từ trình duyệt tới nguồn ngoài rồi quay về ra sao. Chúng ta sẽ đọc thẳng từ mã nguồn để không nói sai một tên lớp hay một endpoint nào.

Các thành phần chính

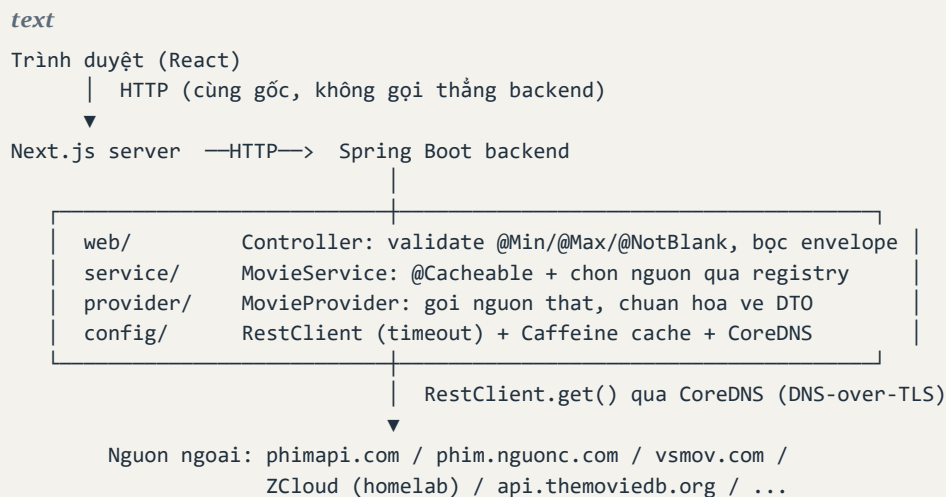
Mã backend phân tầng theo mô hình MVC quen thuộc của Spring, mỗi gói (package) một trách nhiệm rõ ràng:

Gói	Vai trò
web/	Controller: nhận request, validate tham số, bọc kết quả vào envelope. Không chứa nghiệp vụ.
service/	Nghiệp vụ và cache. <code>MovieService</code> chọn nguồn qua <code>ProviderRegistry</code> rồi gọi provider tương ứng.
provider/	Cổng ra ngoài. <code>MovieProvider</code> là interface, mỗi nguồn một cài đặt (kkphim/, nguonc/, vsmov/, homelab/, tmdb/, anilist/, jikan/).
dto/	Schema chuẩn hoá lộ ra REST API: <code>MovieSummary</code> , <code>MovieDetail</code> , <code>Taxonomy</code> , <code>ProviderType</code> ...
common/	Vỏ bọc phản hồi dùng chung: <code>ApiResponse</code> , <code>PageResponse</code> , <code>PageMeta</code> , <code>ApiError</code> .
config/	Cấu hình hạ tầng: <code>CacheConfig</code> , <code>RestClientConfig</code> , <code>ProviderProperties</code> , <code>CORS</code> , <code>OpenAPI</code> .
exception/	<code>GlobalExceptionHandler</code> gom mọi lỗi về một định dạng thống nhất.

Ngoài mã Java, tầng hạ tầng còn có một container CoreDNS (định nghĩa trong `Corefile`) làm bộ phân giải tên miền riêng cho toàn bộ stack Docker.

Sơ đồ các tầng và trách nhiệm

Một lời gọi luôn đi xuống theo đúng bốn tầng, và mỗi tầng chỉ làm việc của mình:



- **Tầng web** (`MovieController`) chỉ nhận request, kiểm tra tham số (`@Min(1)`, `@Max(64)`, `@NotBlank`) rồi gói kết quả vào `ApiResponse`. Nó không biết dữ liệu từ đâu ra.
- **Tầng service** (`MovieService`) quyết định lấy từ nguồn nào và có cache hay không. Nó không tự gọi HTTP; nó nhờ `ProviderRegistry` tìm cài đặt phù hợp.
- **Tầng provider** là nơi duy nhất biết định dạng thật của từng nguồn. Nó gọi HTTP, đọc JSON thô, rồi dịch sang DTO chuẩn.
- **Tầng config** cung cấp công cụ cho các tầng trên: `RestClient` đã cài sẵn timeout, `CacheManager` của Caffeine, và bộ phân giải DNS riêng.

Trừu tượng Provider

Đây là trái tim của kiến trúc. Tầng service chỉ làm việc qua một interface, nhờ đó thêm nguồn mới không đụng tới controller hay service.

Interface `MovieProvider`

`MovieProvider` là hợp đồng chung cho mọi nguồn phim. Trích từ `MovieProvider.java`:

```

java
public interface MovieProvider {

    /** Nguồn mà cài đặt này đại diện. */
    ProviderType type();

    /** Mã định danh dùng trên URL. */
    default String code() {
        return type().code();
    }

    PageResponse<MovieSummary> latest(MovieQuery query);
    PageResponse<MovieSummary> listByType(ListType listType, MovieQuery query);
    PageResponse<MovieSummary> search(String keyword, MovieQuery query);
    PageResponse<MovieSummary> listByCategory(String categorySlug, MovieQuery query);
    PageResponse<MovieSummary> listByCountry(String countrySlug, MovieQuery query);
    PageResponse<MovieSummary> listByYear(int year, MovieQuery query);
    Optional<MovieDetail> findBySlug(String slug);
    List<Taxonomy> categories();
    List<Taxonomy> countries();
}

```

Hai điểm đáng chú ý:

- `type()` trả về nguồn mà cài đặt đại diện, dùng làm khoá tra cứu.
- `code()` là mã xuất hiện trên URL (`?provider=kkphim`). Mặc định nó lấy thẳng mã của

`ProviderType`, nhưng nguồn do người dùng tự thêm — vốn không nằm trong enum — sẽ ghi đè hàm này bằng mã riêng của mình.

Enum `ProviderType`

Các nguồn dựng sẵn được liệt kê trong `ProviderType.java`, mỗi hằng gắn với một mã dùng trên URL:

```

java
public enum ProviderType {
    KKPHIM("kkphim"),    // phimapi.com
    NGUONC("nguonc"),    // phim.nguonc.com
    VSMOV("vsmov"),      // vsmov.com
    HOMELAB("homelab");  // Kho phim riêng tren homelab, doc qua ZCloud
    ...
}

```

Enum cũng đóng vai trò bộ chuyển đổi khi Jackson đọc/ghi JSON: `@JsonValue` xuất ra mã chữ thường, còn `@JsonCreator from(...)` nhận cả mã lẫn tên hằng, giá trị rỗng thì rơi về `KKPHIM` làm mặc định.

`ProviderRegistry` — **tự phát hiện** `@Component` *theo* `type()`

`ProviderRegistry` là chỗ Spring gom mọi cài đặt lại. Điểm mấu chốt: constructor nhận `List<MovieProvider>` — Spring tự tìm **mọi bean nào cài đặt interface này** (mỗi provider được đánh dấu `@Component`) — rồi lập chỉ mục theo `type()`. Trích từ `ProviderRegistry.java`:

```

java
@Component
public class ProviderRegistry {

    private final Map<ProviderType, MovieProvider> providers = new
    EnumMap<>(ProviderType.class);
    private final CustomSourceService customSources;

    public ProviderRegistry(List<MovieProvider> discovered, @Lazy CustomSourceService
    customSources) {
        discovered.forEach(provider -> providers.put(provider.type(), provider));
        this.customSources = customSources;
    }
    ...
}

```

Nhờ cách này, thêm một nguồn dựng sẵn mới chỉ cần viết một lớp `@Component` cài đặt `MovieProvider` và thêm một hằng vào `ProviderType`. Registry sẽ tự nhận, không phải sửa danh sách nào bằng tay.

Việc tra cứu tách làm hai lỗi, thấy rõ trong hàm `get(String code)`:

```

java
public MovieProvider get(String code) {
    if (code == null || code.isBlank()) {
        return get(ProviderType.KKPHIM); // rong -> nguồn mặc định
    }
    String wanted = code.trim().toLowerCase();
    for (ProviderType type : ProviderType.values()) {
        if (type.code().equals(wanted)) {
            return get(type); // nguồn dựng sẵn
        }
    }
    MovieProvider custom = customSources.activeProviders().get(wanted);
    if (custom == null) {
        throw new IllegalArgumentException("Nguồn '" + code + "' không hợp lệ. ...");
    }
    return custom; // nguồn người dựng tự thêm
}

```

Tìm trong các nguồn dựng sẵn trước, rồi mới đến nguồn người dùng tự thêm (nạp lúc chạy từ `CustomSourceService`). Mã sai vẫn ném `IllegalArgumentException` kèm danh sách các nguồn đang có — cuối cùng ra HTTP 400. Tham chiếu `@Lazy` tới `CustomSourceService` là để phá vòng phụ thuộc: dịch vụ nguồn tự thêm dùng lại cách đọc của `KKPhim`, còn registry lại phải hỏi nó để biết có những mã nào.

Chuẩn hoá DTO

Mỗi nguồn trả về JSON một kiểu, nhưng tầng service và tầng web chỉ thấy **một schema chung**. Việc quy đổi nằm gọn trong tầng provider.

`MovieSummary` và `MovieDetail`

Hai DTO trung tâm là `MovieSummary` (bản rút gọn, dùng cho lưới danh sách) và `MovieDetail` (bản đầy đủ, kèm danh sách server và tập). Cả hai đều là *record* bất biến. Ví dụ hàm chuyển đổi trong `KKPhimProvider.java` gom JSON thô của `phimapi.com` về `MovieSummary`:

```
java
```

```
private MovieSummary toSummary(KKPhimModels.Item item, String cdn) {
    return new MovieSummary(
        item.id(),
        item.slug(),
        item.name(),
        item.originName(),
        ProviderSupport.absoluteImage(item.posterUrl(), cdn),
        ProviderSupport.absoluteImage(item.thumbUrl(), cdn),
        item.year(),
        item.type(),
        item.quality(),
        item.lang(),
        item.time(),
        item.episodeCurrent(),
        toTaxonomies(item.category()),
        toTaxonomies(item.country()),
        toTmdb(item.tmdb()),
        toImdb(item.imdb()),
        code,
        item.modified() == null ? null : ProviderSupport.normalizeInstant(item.modified().t
ime()));
}
```

Điểm cần nhớ: dù JSON gốc của KKPhim, NguonC hay VSMOV khác nhau ra sao, chúng đều được dồn về đúng các trường này. `MovieSummary` còn có hàm tĩnh `from(MovieDetail)` để rút một bản chi tiết thành mục danh sách khi cần.

Vai trò của `ProviderSupport`

`ProviderSupport` là bộ hàm tiện ích dùng chung cho **mọi provider** khi dịch dữ liệu thô sang DTO. Nó gom những phép chuẩn hoá mà nguồn nào cũng cần, để không nơi nào phải chép lại. Trích từ `ProviderSupport.java`:

Hàm	Việc nó làm
<code>slugify(value)</code>	Sinh slug từ tên tiếng Việt ("Hành Động" thành "hanh-dong"), dùng cho nguồn chỉ trả về tên mà không trả slug.
<code>absoluteImage(path, cdn)</code>	Ghép đường dẫn ảnh tương đối với gốc CDN; giữ nguyên nếu đã là URL tuyệt đối.
<code>isPlaceholder(value)</code>	Nhận diện giá trị rỗng kiểu "Đang cập nhật" / "updating", so trên slug nên khớp cả có dấu lẫn không dấu.
<code>splitCsv(value)</code>	Tách chuỗi ngăn bằng dấu phẩy, bỏ phần tử rỗng và phần tử placeholder.
<code>normalizeInstant(raw)</code>	Đưa chuỗi thời gian về ISO-8601 chuẩn, trả nguyên bản nếu không đọc được.
<code>parseInt(raw)</code>	Chuyển chuỗi sang số nguyên an toàn, trả <code>null</code> nếu không hợp lệ.
<code>orEmpty(list)</code>	Trả danh sách rỗng thay vì <code>null</code> , tránh <code>NullPointerException</code> khi nguồn thiếu trường.

Nhờ tập hàm này, mỗi provider chỉ còn phải lo phần khác biệt thật sự của nguồn mình.

Vỏ bọc phản hồi và phân trang

Mọi phản hồi REST đều đi qua hai vỏ bọc chung, giúp client xử lý nhất quán bất kể endpoint.

ApiResponse — *envelope chuẩn*

ApiResponse.java là một record bọc mọi kết quả thành công lẫn thất bại:

java

```
public record ApiResponse<T>(boolean success, String message, T data, Instant timestamp) {
    public static <T> ApiResponse<T> ok(T data) { return new ApiResponse<>(true, "OK",
        data, Instant.now()); }
    public static <T> ApiResponse<T> fail(String message) { return new ApiResponse<>(false,
        message, null, Instant.now()); }
}
```

Hình dạng JSON quen thuộc client luôn nhận được:

json

```
{
  "success": true,
  "message": "OK",
  "data": { },
  "timestamp": "2026-09-08T16:20:00Z"
}
```

PageResponse — *kết quả có phân trang*

Kết quả dạng danh sách nằm trong PageResponse.java, kèm khối meta phân trang và trường provider cho biết nguồn nào đã phục vụ:

java

```
public record PageResponse<T>(List<T> items, PageMeta meta, String provider) {
    public static <T> PageResponse<T> of(List<T> items, PageMeta meta, String provider) {
        return new PageResponse<>(items == null ? List.of() : items, meta, provider);
    }
    public static <T> PageResponse<T> empty(int page, int limit, String provider) {
        return new PageResponse<>(List.of(), PageMeta.of(page, limit, 0), provider);
    }
}
```

Hàm empty(...) được dùng khi nguồn trả rỗng (ví dụ 404 kèm body JSON), để trả một trang rỗng hợp lệ thay vì ném lỗi. Trong lời gọi danh sách, controller lồng cả hai vỏ: ApiResponse<PageResponse<MovieSummary>>.

Cache Caffeine

Gọi nguồn ngoài là phần chậm và dễ hỏng nhất, nên kết quả được đệm trong bộ nhớ bằng Caffeine. Cấu hình nằm trong CacheConfig.java: mỗi loại dữ liệu một cache riêng, thời gian sống và dung lượng chọn theo mức độ đổi thay của dữ liệu.


```

java
@Configuration
@EnableCaching
public class CacheConfig {

    public static final String MOVIE_LIST_CACHE      = "movieLists";
    public static final String MOVIE_DETAIL_CACHE    = "movieDetails";
    public static final String TAXONOMY_CACHE        = "taxonomies";
    public static final String HOMELAB_LIBRARY_CACHE = "homelabLibrary";

    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager manager = new CaffeineCacheManager();
        manager.registerCustomCache(MOVIE_LIST_CACHE, Caffeine.newBuilder()
            .maximumSize(500).expireAfterWrite(Duration.ofMinutes(5)).build());
        // ... cac cache con lai
        return manager;
    }
}

```

Bảng tóm tắt bốn cache:

Cache (hàng)	Tên	Thời gian sống	Dung lượng	Lý do
MOVIE_LIST_CACHE	movieLists	5 phút	500 mục	Danh sách đổi nhanh nên giữ ngắn.
MOVIE_DETAIL_CACHE	movieDetails	30 phút	1.000 mục	Chi tiết ổn định hơn, giữ lâu hơn.
TAXONOMY_CACHE	taxonomies	12 giờ	32 mục	Thể loại / quốc gia gần như tĩnh.
HOMELAB_LIBRARY_CACHE	homelabLibrary	3 phút	4 mục	Duyệt cả cây file kho riêng nên bắt buộc phải đệm; giữ ngắn để chép phim mới vào là thấy ngay.

Cache được kích hoạt ở tầng service. `MovieService` gắn `@Cacheable` lên từng phương thức, lấy khoá từ chính tham số truy vấn — trích từ `MovieService.java`:

```

java
@Cacheable(cacheNames = CacheConfig.MOVIE_LIST_CACHE,
    key = "'latest:' + #provider + ':' + #query.page() + ':' + #query.limit()")
public PageResponse<MovieSummary> latest(String provider, MovieQuery query) {
    return resolve(provider).latest(query);
}

@Cacheable(cacheNames = CacheConfig.MOVIE_DETAIL_CACHE, key = "#provider + ':' + #slug")
public MovieDetail findBySlug(String provider, String slug) {
    return resolve(provider).findBySlug(slug)
        .orElseThrow(() -> new ResourceNotFoundException("MOVIE_NOT_FOUND", "..."));
}

```

Khoá cache luôn có tiền tố `provider` để hai nguồn không đè kết quả của nhau. Xem trạng thái các cache tại <http://localhost:8080/actuator/caches>.

Cấu hình RestClient và timeout

Mọi lời gọi ra ngoài đi qua `RestClient` của Spring. Các bean được dựng sẵn trong `RestClientConfig.java`, mỗi nguồn một client riêng, tất cả dùng chung một factory có gắn timeout:

```
java
@Bean
public ClientHttpRequestFactory providerRequestFactory() {
    HttpClientSettings settings = HttpClientSettings.defaults()
        .withTimeouts(properties.connectTimeout(), properties.readTimeout());
    return ClientHttpRequestFactoryBuilder.detect().build(settings);
}

@Bean
public RestClient kkphimRestClient(RestClient.Builder builder, ClientHttpRequestFactory
    factory) {
    return baseClient(builder, factory, properties.kkphim().baseUrl());
}
```

Giá trị timeout đến từ `ProviderProperties` (tiền tố `rapphim.provider`) và được đặt trong `application.yml`:

Tham số	Giá trị mặc định	Ý nghĩa
<code>rapphim.provider.connect-timeout</code>	5s	Thời gian tối đa để mở kết nối tới nguồn.
<code>rapphim.provider.read-timeout</code>	20s	Thời gian tối đa chờ một phản hồi.

Mục đích của timeout: một nguồn chậm sẽ không kéo treo cả API. Khi `RestClient` ném lỗi, provider bắt lại và chuyển thành `UpstreamException` — cuối cùng ra HTTP 502 `UPSTREAM_ERROR` cho client, thay vì để request treo vô hạn. Xem cách bắt lỗi trong `KKPhimProvider.call(...)`:

```
java
try {
    return client.get().uri(uriFunction::apply).retrieve()
        .onStatus(HttpStatus::is4xxClientError, (request, response) -> { /* doc body
            nhu du lieu rong */ })
        .body(responseType);
} catch (RestClientException ex) {
    log.warn("Goi KKPhim that bai: {}", ex.getMessage());
    throw new UpstreamException(code, "Khong lay duoc du lieu tu KKPhim: " + ex.getMessage(),
        ex);
}
```

Mỗi client còn được gắn sẵn header `User-Agent` và `Accept: application/json`. Riêng client cho TMDb, AniList, Jikan và ZCloud (kho riêng) **không** gắn sẵn thông tin xác thực ở tầng này: khoá đối được lúc chạy trong khi client chỉ dựng một lần lúc khởi động, nên token/khoá được gắn vào từng request ở lớp client tương ứng.

CoreDNS với DNS-over-TLS

Stack Docker có thêm một container CoreDNS. Đây không phải chi tiết trang trí, mà là cách xử lý một vấn đề mạng có thật.

Tại sao cần

Trên một số mạng (đo thật trên máy chủ homelab), nhà mạng **chèn phản hồi DNS giả** cho vài tên miền. Hỏi `api.themoviedb.org` năm lần bằng DNS thường thì ba lần trả về `127.0.0.1`. Backend gọi vào địa chỉ đó và nhận lỗi `I/O error`, trông như mã hỏng, thực ra là không bao giờ kết nối tới đúng nơi.

DNS-over-TLS đưa truy vấn DNS đi trong một đường mã hoá, nên nhà mạng không đọc được tên miền để mà chèn phản hồi giả. Đổi lại, ta phải tự chạy một bộ phân giải — và đó chính là lý do container CoreDNS tồn tại. Cấu hình trong `Corefile`:

```
text
.:53 {
  forward . tls://8.8.8.8 tls://8.8.4.4 {
    tls_servername dns.google
  }

  # Dem lai cho do phai mo ket noi TLS moi lan hoi.
  cache 300

  errors
}
```

CoreDNS nhận truy vấn DNS thường ở cổng 53 từ các container khác, rồi chuyển tiếp qua đường TLS tới `8.8.8.8 / 8.8.4.4` với tên máy chủ `dns.google`. Khối `cache 300` đệm kết quả 300 giây để đỡ phải mở kết nối TLS mới mỗi lần hỏi.

Trong `docker-compose.yml`, container `dns` được gán địa chỉ IP tĩnh `172.29.0.53`, và backend trở tới nó bằng chính IP đó (`dns: [172.29.0.53]`) — ở mức này Docker không nhận tên dịch vụ, buộc phải ghi bằng IP.

Lưu ý phạm vi: DNS-over-TLS chỉ chống được việc **chèn phản hồi DNS giả**. Nếu mạng còn chặn theo tên miền ngay trong bắt tay TLS (như trường hợp `api.themoviedb.org` trên homelab), thì DNS riêng không đủ — phải cho lưu lượng đi vòng ra ngoài bằng VPN hoặc tunnel. Đây là giới hạn của mạng, không sửa được từ trong mã nguồn.

Chi tiết triển khai, số liệu đo được và các phép kiểm mạng liên quan nằm trong tài liệu triển khai chính, mục *Bộ phân giải tên miền riêng* của README dự án.

Luồng một request thực tế

Để thấy các tầng ăn khớp ra sao, hãy theo một lời gọi thật: người dùng mở trang chủ, cần danh sách phim mới cập nhật từ nguồn `kkphim`.

1. **Trình duyệt** render trang chủ. Vì các trang là Server Component của Next.js, lời gọi

dữ liệu chạy trên **máy chủ Next**, không phải trong trình duyệt — nên trình duyệt không gọi thẳng backend và không cần CORS cho luồng chính.

1. **Máy chủ Next** gọi backend qua địa chỉ nội bộ (`API_INTERNAL_URL`, ví dụ

`http://backend:8080`): `GET /api/v1/movies/latest?page=1&limit=24&provider=kkphim`.

1. `MovieController.latest(...)` nhận request. Spring validate tham số bằng annotation

(`@Min(1) int page, @Min(1) @Max(64) int limit`), dựng `MovieQuery.of(page, limit)` rồi gọi tầng service.

1. `MovieService.latest("kkphim", query)` được đánh dấu `@Cacheable` trên cache

`movieLists` với khoá `"latest:kkphim:1:24"`. Nếu khoá đã có, kết quả trả về ngay, không đi tiếp bước nào.

1. Nếu cache trượt, service gọi `resolve("kkphim")`, tức

`ProviderRegistry.get("kkphim")`, trả về bean `KKPhimProvider`.

1. `KKPhimProvider.latest(query)` dựng URL

`/v1/api/danh-sach/phim-moi-cap-nhat?page=1&limit=24&sort_field=modified.time&sort_type=desc` rồi gọi qua `kkphimRestClient` (đã cài timeout 5s / 20s).

1. **Phân giải tên miền:** để mở kết nối tới `phimapi.com`, container backend hỏi DNS qua

CoreDNS (172.29.0.53), CoreDNS chuyển tiếp bằng DNS-over-TLS và trả về địa chỉ IP thật, không bị chèn giả.

1. **Nguồn KKPhim** trả về JSON thô. Provider dịch mỗi phần tử sang `MovieSummary` bằng

`toSummary(...)` (ghép ảnh với CDN, chuẩn hoá thời gian, gom thể loại/quốc gia), rồi gói vào `PageResponse.of(items, meta, "kkphim")`.

1. Kết quả quay ngược lên `MovieService`, được Caffeine ghi vào cache `movieLists` dưới

khoá bước 4 để 5 phút tới không phải gọi lại nguồn.

1. `MovieController` bọc `PageResponse` vào `ApiResponse.ok(...)` và trả HTTP 200:

`ApiResponse<PageResponse<MovieSummary>>`.

1. **Máy chủ Next** nhận JSON, render HTML trang chủ rồi trả về **trình duyệt**. Người

dùng thấy lưới thẻ phim; nếu một tập nào đó cần chi tiết, chu trình lặp lại với endpoint `GET /api/v1/movies/{slug}` và cache `movieDetails`.

Nếu nguồn lỗi hoặc timeout ở bước 6–8, provider ném `UpstreamException`, `GlobalExceptionHandler` gom về `ApiError` với code = `UPSTREAM_ERROR` và HTTP 502 — client nhận lỗi có cấu trúc thay vì một request treo.

Kết luận

Kiến trúc RapPhim Warehouse xoay quanh một ý tưởng: **cô lập sự khác biệt của các nguồn vào tầng provider**, để mọi tầng phía trên chỉ làm việc với một schema DTO chung.

- Tầng web validate và bọc envelope (`ApiResponse`, `PageResponse`), không chứa nghiệp vụ.
- Tầng service chọn nguồn qua `ProviderRegistry` và đệm kết quả bằng Caffeine.
- Tầng provider — mỗi nguồn một `@Component` cài đặt `MovieProvider`, được registry tự phát

hiện theo `type()` — là nơi duy nhất biết định dạng thật, dịch về DTO nhờ `ProviderSupport`.

- Tầng config cung cấp `RestClient` có timeout, cache, và một bộ phân giải CoreDNS

DNS-over-TLS để vượt qua việc nhà mạng chèn phản hồi DNS giả.

Nhờ vậy, thêm một nguồn phim mới chỉ cần một lớp cài đặt `MovieProvider` và một hằng trong `ProviderType` — controller và service không phải sửa.

Tài liệu liên quan trong bộ:

- README dự án — tổng quan, danh sách endpoint, và tài liệu triển khai

(bao gồm mục *Bộ phân giải tên miền riêng* giải thích sâu về CoreDNS).

- `docs/openapi.json` — đặc tả OpenAPI 3.1 sinh tự động, dùng để tra cứu chính xác từng endpoint và schema.
- Từ Commit Đến Container — sổ tay sự cố DevOps rút từ chính nhật ký triển khai dự án (Git, Docker, CI/CD).

Hướng dẫn thêm một nguồn phim (Provider) mới

Tài liệu

Phiên bản: 1.0

Cập nhật: 2026-09-11

Đối tượng đọc: Lập trình viên backend mở rộng nguồn phim

Trạng thái: Chính thức

Tag: provider, backend, spring-boot, huong-dan

Giới thiệu

Backend của RapPhim Warehouse là một **lớp trung gian**: nó gọi nhiều nguồn phim khác nhau, chuẩn hoá dữ liệu thô về **một schema DTO duy nhất**, cache lại bằng Caffeine rồi phục vụ cho frontend. Nhờ vậy frontend chỉ cần biết một dạng dữ liệu, còn việc mỗi nguồn trả JSON một kiểu là chuyện của tầng provider.

Tài liệu này hướng dẫn **từng bước thêm một nguồn phim mới ở mức mã nguồn** — nghĩa là nguồn có cấu trúc API riêng, cần một lớp Java biên dịch cùng ứng dụng. Đây là loại việc quan trọng nhất khi mở rộng kho phim, nên chúng ta đi kỹ và kèm ví dụ trích từ hai nguồn đã có: **NguonC** (phim.nguonc.com) và **VSMOV** (vsmov.com).

Có hai cách thêm nguồn, đừng nhầm lẫn:

- Nguồn cấu hình lúc chạy** (runtime): các nguồn nói *đúng giọng KKPhim*, chỉ khác `base-url`. Quản trị viên thêm ngay trên trang quản trị, `CustomSourceService` lo phần còn lại. Không cần đụng mã nguồn.
- Nguồn ở mức mã nguồn** (tài liệu này): nguồn có **cấu trúc phản hồi riêng** — vỏ bọc khác, tên trường khác, kiểu dữ liệu lệch — nên phải viết một lớp `MovieProvider` riêng, biên dịch cùng ứng dụng.

Xuyên suốt tài liệu, chúng ta lấy một nguồn ví dụ giả định tên **OPhim** (mã `ophim`) — một API kiểu PHP giống VSMOV/NguonC — để minh hoạ. Hãy thay `ophim/OPhim/Ophim` bằng nguồn thật của bạn.

Các thành phần chính

Trước khi viết, cần nắm các mảnh ghép và vai trò của chúng:

Thành phần	Đường dẫn	Vai trò
MovieProvider	<code>provider/MovieProvider.java</code>	Hợp đồng chung mọi nguồn phải cài đặt. Controller chỉ làm việc qua interface này.
ProviderRegistry	<code>provider/ProviderRegistry.java</code>	Tự phát hiện mọi bean <code>MovieProvider</code> và định tuyến theo mã trên URL.
ProviderType	<code>dto/ProviderType.java</code>	Enum liệt kê các nguồn dựng sẵn kèm mã dùng trên URL.
ProviderProperties	<code>config/ProviderProperties.java</code>	Cấu hình <code>base-url</code> , CDN ảnh, timeout — đọc từ tiền tố <code>rapphim.provider</code> .

Thành phần	Đường dẫn	Vai trò
RestClientConfig	config/RestClientConfig.java	Tạo sẵn một <code>RestClient</code> riêng cho mỗi nguồn, kèm timeout và header mặc định.
ProviderSupport	provider/ProviderSupport.java	Hàm tiện ích dùng chung: <code>slugify</code> , <code>absoluteImage</code> , <code>normalizeInstant</code> , <code>parseInt</code> , <code>orEmpty...</code>
Model records	provider/<nguồn>/model/...	Các record Jackson ánh xạ 1-1 với JSON thô của nguồn.
DTO chuẩn	dto/MovieSummary.java, dto/MovieDetail.java, dto/Taxonomy.java	Schema thống nhất mà mọi nguồn phải chuyển về.

Điểm cần nhớ đầu tiên: `ProviderRegistry` **tự phát hiện nguồn**. Hãy nhìn constructor của nó:

```
java
public ProviderRegistry(List<MovieProvider> discovered, @Lazy CustomSourceService
    customSources) {
    discovered.forEach(provider -> providers.put(provider.type(), provider));
    this.customSources = customSources;
}
```

Spring tiêm vào `discovered` **mọi** bean cài đặt `MovieProvider`, rồi registry lập chỉ mục theo `ProviderType` mà `type()` trả về. Do đó, **chỉ cần đánh dấu lớp mới bằng `@Component`** là nó tự vào registry — không có bảng đăng ký thủ công nào phải sửa.

Trước khi bắt đầu

Hãy khảo sát nguồn mới bằng `curl` để biết chính xác hình dạng JSON: vỏ bọc nằm ở đâu (`items` ở gốc hay lồng trong `data`?), tên trường phân trang, ảnh là URL tuyệt đối hay đường dẫn tương đối, và các trường nào có thể **thiếu**. Chính những khác biệt này quyết định bộ model bạn phải viết.

```
bash
# Khảo sát endpoint danh sách của nguồn mới trước khi viết model
curl -s "https://ophim.example/api/danh-sach/phim-moi-cap-nhat?page=1" | head -c 2000
```

Bước 1 — Thêm giá trị vào enum `ProviderType`

Mở `dto/ProviderType.java` và thêm một hằng enum kèm mã dùng trên URL. Mã này chính là chuỗi client gửi lên (`?provider=ophim`).

```
java
public enum ProviderType {

    /** KKPhim - phimapi.com */
    KKPHIM("kkphim"),

    /** NguonC - phim.nguonc.com */
    NGUONC("nguonc"),

    /** VSMOV - vsmov.com */
    VSMOV("vsmov"),

    /** Kho phim riêng tren homelab, doc qua ZCloud. */
    HOMELAB("homelab"),

    /** OPhim - ophim.example (nguồn ví dụ thêm mới). */
    OPHIM("ophim");

    // ... phần còn lại giữ nguyên
}
```

Lưu ý: `ProviderType.from(String)` ném lỗi kèm danh sách mã hợp lệ được **viết cứng** trong thông báo:

```
java
.orElseThrow(() -> new IllegalArgumentException(
    "Nguồn '" + value + "' không hợp lệ. Chỉ chấp nhận: kkphim, nguonc, vsmov, homelab"));
```

Hãy cập nhật chuỗi này để bổ sung `ophim`, nếu không thông báo lỗi sẽ lạc hậu.

Sau bước này, `ProviderRegistry.get(String)` đã định tuyến được URL có `provider=ophim`: nó duyệt `ProviderType.values()`, thấy mã khớp và trả về bean tương ứng (bean đó ta tạo ở Bước 5). Vì `type()` nằm trong enum, ta **không cần** ghi đè `code()` — mặc định `code()` trả về `type().code()`.

Bước 2 — Khai báo cấu hình trong `ProviderProperties` và `application.yml`

`ProviderProperties` là một record gắn với tiền tố `rapphim.provider`. Thêm một thành phần `Endpoint` cho nguồn mới, và đặt giá trị mặc định trong compact constructor để ứng dụng vẫn chạy khi cấu hình để trống.


```
java
```

```
@ConfigurationProperties(prefix = "rapphim.provider")
public record ProviderProperties(
    Endpoint kkphim,
    Endpoint nguonc,
    Endpoint vsmov,
    Endpoint ophim,          // them moi
    Duration connectTimeout,
    Duration readTimeout
) {

    public ProviderProperties {
        kkphim = kkphim == null ? new Endpoint("https://phimapi.com", "https://phimimg.com") :
        kkphim;
        nguonc = nguonc == null ? new Endpoint("https://phim.nguonc.com", "") : nguonc;
        vsmov = vsmov == null ? new Endpoint("https://vsmov.com", "") : vsmov;
        // Neu anh cua OPhim la URL tuyet doi thi de goc CDN trong ("" ).
        ophim = ophim == null ? new Endpoint("https://ophim.example", "") : ophim;
        connectTimeout = connectTimeout == null ? Duration.ofSeconds(5) : connectTimeout;
        readTimeout = readTimeout == null ? Duration.ofSeconds(20) : readTimeout;
    }

    public record Endpoint(String baseUrl, String cdnImage) {
    }
}
```

Trường `cdnImage` chỉ cần khi nguồn trả **đường dẫn ảnh tương đối** (như KKPhim). Nếu nguồn trả URL tuyệt đối (như VSMOV), để trống — `ProviderSupport.absoluteImage(...)` sẽ giữ nguyên URL đó.

Không cần khai báo `@EnableConfigurationProperties`: `WarehouseApplication` đã bật `@ConfigurationPropertiesScan`, mọi lớp `@ConfigurationProperties` được nạp tự động.

Tiếp theo, khai báo giá trị trong `application.yml` dưới `rapphim.provider`:

```
yaml
```

```
rapphim:
  provider:
    connect-timeout: 5s
    read-timeout: 20s
    ophim:
      base-url: https://ophim.example
      # Anh cua OPhim la URL tuyet doi nen khong can goc CDN.
      cdn-image: ""
```

Bảng tham chiếu cấu hình:

Khoá	Ý nghĩa	Ví dụ
<code>rapphim.provider.ophim.base-url</code>	Gốc REST API của nguồn	<code>https://ophim.example</code>
<code>rapphim.provider.ophim.cdn-image</code>	Gốc CDN ảnh (để trống nếu ảnh là URL tuyệt đối)	<code>""</code>
<code>rapphim.provider.connect-timeout</code>	Thời gian tối đa để mở kết nối (dùng chung)	<code>5s</code>
<code>rapphim.provider.read-timeout</code>	Thời gian tối đa chờ một phản hồi (dùng chung)	<code>20s</code>

Lưu ý cho test: thêm một thành phần vào `ProviderProperties` sẽ làm các lời gọi constructor rút gọn trong test bị thiếu tham số. Ví dụ `VsmovProviderTest` dựng `new ProviderProperties(null, null, null, null, null);` sau khi thêm `ophim`, phải bổ sung một `null` nữa. Trình biên dịch sẽ nhắc bạn.

Bước 3 — Tạo các record model Jackson

Tạo gói `provider/ophim/model` và một lớp `OphimModels` gói các record ánh xạ 1-1 với JSON thô. Mỗi record đánh dấu `@JsonIgnoreProperties(ignoreUnknown = true)` để bỏ qua trường lạ, và dùng `@JsonProperty` khi tên JSON là `snake_case`.

CẢNH BÁO — API kiểu PHP có thể trả `{}` (object rỗng) cho trường thiếu. Đây là bài học thật từ VSMOV: trường `poster_url/thumb_url` từng được khai báo là `String`, nhưng nguồn trả về `{}` khi thiếu ảnh. Jackson gặp object ở nơi mong đợi chuỗi liền ném `MismatchedInputException` và làm hỏng cả phản hồi. Cách xử lý: khai báo kiểu `Object` cho các trường có nguy cơ này, rồi ép về chuỗi bằng một helper chỉ nhận khi *thực sự là chuỗi*. **Không dùng String.**

Trích chính model của VSMOV — chú thích ngay trên trường ghi rõ vì sao:

```
java
@JsonIgnoreProperties(ignoreUnknown = true)
public record Item(
    @JsonProperty("_id") String id,
    String name,
    String slug,
    @JsonProperty("origin_name") String originName,
    // VSMOV tra {} (object rỗng) cho ảnh thiếu thay vì chuỗi/null - de Object roi
    // loc trong provider, neu de String se vo khi gap {}.
    @JsonProperty("poster_url") Object posterUrl,
    @JsonProperty("thumb_url") Object thumbUrl,
    Integer year,
    TmdbRaw tmdb,
    ImdbRaw imdb,
    Modified modified) {
}
```

Áp dụng đúng khuôn cho nguồn mới. Khai báo vỏ bọc danh sách, phân trang, chi tiết và các khối con. Ví dụ khung `OphimModels`:

java

```
public final class OphimModels {

    private OphimModels() {
    }

    /** Vo boc danh sach. Bo qua truong khong dung (status, pathImage...). */
    @JsonIgnoreProperties(ignoreUnknown = true)
    public record ListEnvelope(List<Item> items, Pagination pagination) {
    }

    @JsonIgnoreProperties(ignoreUnknown = true)
    public record Pagination(Integer totalItems, Integer totalItemsPerPage,
                             Integer currentPage, Integer totalPages) {
    }

    @JsonIgnoreProperties(ignoreUnknown = true)
    public record Item(
        @JsonProperty("_id") String id,
        String name,
        String slug,
        @JsonProperty("origin_name") String originName,
        // Truong anh CO THE la {} - khai bao Object, ep chuoai trong provider.
        @JsonProperty("poster_url") Object posterUrl,
        @JsonProperty("thumb_url") Object thumbUrl,
        Integer year) {
    }

    /** Vo boc chi tiet: mot phim + danh sach server phat. */
    @JsonIgnoreProperties(ignoreUnknown = true)
    public record DetailEnvelope(Movie movie, List<ServerRaw> episodes) {
    }

    @JsonIgnoreProperties(ignoreUnknown = true)
    public record Movie(
        @JsonProperty("_id") String id,
        String name,
        String slug,
        @JsonProperty("origin_name") String originName,
```

java (tiếp)

```
String content,
@JsonProperty("poster_url") Object posterUrl,
@JsonProperty("thumb_url") Object thumbUrl,
Integer year,
String type,
String status,
@JsonProperty("episode_current") String episodeCurrent,
List<TaxonomyItem> category,
List<TaxonomyItem> country) {
}

@JsonIgnoreProperties(ignoreUnknown = true)
public record ServerRaw(
    @JsonProperty("server_name") String serverName,
    @JsonProperty("server_data") List<EpisodeRaw> serverData) {
}

@JsonIgnoreProperties(ignoreUnknown = true)
public record EpisodeRaw(
    String name,
    String slug,
    @JsonProperty("link_embed") String linkEmbed,
    @JsonProperty("link_m3u8") String linkM3u8) {
}

@JsonIgnoreProperties(ignoreUnknown = true)
public record TaxonomyItem(String name, String slug) {
}
}
```

Nếu nguồn gom phân loại theo *kiểu NguồnC* (một Map khoá bằng số thứ tự, mỗi phần tử có *group + list*) thì mô phỏng `NguồnCModels.CategoryGroup/GroupInfo` thay cho `List<TaxonomyItem>`. Luôn để bộ model **nội bộ** trong tầng provider, không để lộ ra REST API.

Bước 4 — Tạo `RestClient` bean trong `RestClientConfig`

Mỗi nguồn có một `RestClient` riêng, chia sẻ chung một `ClientHttpRequestFactory` đã đặt timeout. Thêm một `@Bean` cho nguồn mới, tái dùng helper `baseClient(...)` sẵn có:

```
java
@Bean
public RestClient ophimRestClient(RestClient.Builder builder, ClientHttpRequestFactory
    factory) {
    return baseClient(builder, factory, properties.ophim().baseUrl());
}
}
```

`baseClient(...)` đã gán `base-url`, `User-Agent` và `Accept: application/json`:

```
java
```

```
private RestClient baseClient(RestClient.Builder builder, ClientHttpRequestFactory factory,
    String baseUrl) {
    return builder.clone()
        .requestFactory(factory)
        .baseUrl(baseUrl)
        .defaultHeader(HttpHeaders.USER_AGENT, USER_AGENT)
        .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
        .build();
}
```

Không cần sửa constructor của `RestClientConfig`: nó đã nhận sẵn `ProviderProperties`, và `properties.ophim()` là thành phần ta vừa thêm ở Bước 2.

Lưu ý Spring 7 (đi kèm Spring Boot 4): từ Spring Framework 7, `HttpHeaders` không còn implements `Map<String, List<String>>`. Đoạn `baseClient` trên vẫn an toàn vì `HttpHeaders.USER_AGENT/HttpHeaders.ACCEPT` chỉ là các hằng `String` tên header. Nhưng nếu bạn viết interceptor để nhào nặn header, hãy dùng chính API của `HttpHeaders` (`set`, `add`, `forEach`, `headerNames()`) — **đừng** gọi các phương thức của `Map` như `get(key)`, `entrySet()`, `putAll()`; chúng không còn nữa và sẽ không biên dịch.

Bước 5 — Cài đặt `MovieProvider`

Đây là phần lõi: đọc JSON từ nguồn, **map sang DTO chuẩn**. Tạo lớp `OphimProvider` đánh dấu `@Component`, tiêm `RestClient` bằng `@Qualifier` đúng tên bean ở Bước 4, và trả `ProviderType.OPHIM` ở `type()`.

Hợp đồng cần cài đặt (từ `MovieProvider`):

Phương thức	Trả về	Ví dụ endpoint (mô phỏng VSMOV)
<code>type()</code>	<code>ProviderType</code>	—
<code>latest(query)</code>	<code>PageResponse<MovieSummary></code>	<code>/api/danh-sach/phim-moi-cap-nhat</code>
<code>listByType(listType, query)</code>	<code>PageResponse<MovieSummary></code>	<code>/api/danh-sach/{slug}</code>
<code>search(keyword, query)</code>	<code>PageResponse<MovieSummary></code>	<code>/api/tim-kiem?keyword=</code>
<code>listByCategory(slug, query)</code>	<code>PageResponse<MovieSummary></code>	<code>/api/the-loai/{slug}</code>
<code>listByCountry(slug, query)</code>	<code>PageResponse<MovieSummary></code>	<code>/api/quoc-gia/{slug}</code>
<code>listByYear(year, query)</code>	<code>PageResponse<MovieSummary></code>	<code>/api/nam/{year}</code>
<code>findBySlug(slug)</code>	<code>Optional<MovieDetail></code>	<code>/api/phim/{slug}</code>
<code>categories()</code>	<code>List<Taxonomy></code>	<code>/api/the-loai</code>
<code>countries()</code>	<code>List<Taxonomy></code>	<code>/api/quoc-gia</code>

Khung lớp provider và các endpoint (trích lược, mô phỏng `VsmovProvider`):

java

```
@Component
public class OphimProvider implements MovieProvider {

    private static final Logger log = LoggerFactory.getLogger(OphimProvider.class);
    private static final String PROVIDER_CODE = "ophim";

    private final RestClient client;
    private final String cdnImage;

    public OphimProvider(@Qualifier("ophimRestClient") RestClient client, ProviderProperties
properties) {
        this.client = client;
        this.cdnImage = properties.ophim().cdnImage();
    }

    @Override
    public ProviderType type() {
        return ProviderType.OPHIM;
    }

    @Override
    public PageResponse<MovieSummary> latest(MovieQuery query) {
        return fetchList("/api/danh-sach/phim-moi-cap-nhat", query, Function.identity());
    }

    @Override
    public PageResponse<MovieSummary> search(String keyword, MovieQuery query) {
        return fetchList("/api/tim-kiem", query, builder -> builder.queryParam("keyword",
keyword));
    }

    @Override
    public Optional<MovieDetail> findBySlug(String slug) {
        OphimModels.DetailEnvelope envelope = call(
            builder -> builder.path("/api/phim/{slug}").build(slug),
            OphimModels.DetailEnvelope.class);
        if (envelope == null || envelope.movie() == null) {
            return Optional.empty();
        }
    }
}
```

java (tiếp)

```
    }
    return Optional.of(toDetail(envelope, slug));
}

// ... listByType / listByCategory / listByCountry / listByYear / categories / countries
}
```

Phần đọc danh sách gom vào một hàm `fetchList` dùng chung — trả `PageResponse.empty(...)` khi nguồn không có items, tránh `NullPointerException`:

java

```
private PageResponse<MovieSummary> fetchList(String path, MovieQuery query,
                                             Function<UriBuilder, UriBuilder> extraParams) {
    OphimModels.ListEnvelope response = call(builder -> {
        UriBuilder uri = builder.path(path).queryParam("page", query.page());
        return extraParams.apply(uri).build();
    }, OphimModels.ListEnvelope.class);

    if (response == null || response.items() == null) {
        return PageResponse.empty(query.page(), query.limit(), PROVIDER_CODE);
    }

    List<MovieSummary> items = response.items().stream().map(this::toSummary).toList();
    return PageResponse.of(items, toMeta(response.pagination(), query), PROVIDER_CODE);
}
```

Bước **map sang DTO** là nơi bài học {} phát huy tác dụng. Dùng helper `asUrl(...)` (trích nguyên từ VSMOV) để **chỉ nhận khi là chuỗi thật**, rồi mới ghép CDN qua `ProviderSupport.absoluteImage`:

java

```
/** OPhim doi khi tra {} (object rong) cho anh thieu; chi nhan khi la chuoi that. */
private static String asUrl(Object value) {
    return value instanceof String url && !url.isBlank() ? url : null;
}

private MovieSummary toSummary(OphimModels.Item item) {
    // Thu tu tham so trung khop record MovieSummary; truong nguon khong co -> null hoac
    List.of().
    return new MovieSummary(
        item.id(),
        item.slug(),
        item.name(),
        item.originName(),
        ProviderSupport.absoluteImage(asUrl(item.posterUrl()), cdnImage),
        ProviderSupport.absoluteImage(asUrl(item.thumbUrl()), cdnImage),
        item.year(),
        null,           // type
        null,           // quality
        null,           // lang
        null,           // time
        null,           // episodeCurrent
        List.of(),      // categories
        List.of(),      // countries
        null,           // tmdb
        null,           // imdb
        PROVIDER_CODE,
        null);          // modifiedAt
}
```

Với chi tiết phim, dựng `MovieDetail` theo đúng thứ tự 24 trường của record. Danh sách server/tập dùng `EpisodeServer + Episode(name, slug, filename, linkEmbed, linkM3u8)`; nếu nguồn chỉ có link embed (như NguồnC/VSMOV) thì để `linkM3u8` là null:

```
java
```

```
private List<EpisodeServer> toServers(List<OphimModels.ServerRaw> servers) {
    return ProviderSupport.orEmpty(servers).stream()
        .map(server -> new EpisodeServer(
            server.serverName(),
            ProviderSupport.orEmpty(server.serverData()).stream()
                .map(ep -> new Episode(ep.name(), ep.slug(), null,
                    ep.linkEmbed(), ep.linkM3u8()))
                .toList()))
        .toList();
}

private List<Taxonomy> toTaxonomies(List<OphimModels.TaxonomyItem> items) {
    // Nguồn này không có id chuỗi cho phân loại -> dùng slug làm ca id (giống VSMOV).
    return ProviderSupport.orEmpty(items).stream()
        .filter(item -> item.slug() != null && !item.slug().isBlank())
        .map(item -> new Taxonomy(item.slug(), item.name(), item.slug()))
        .toList();
}
```

Tập dụng `ProviderSupport`: `slugify` (sinh slug từ tên tiếng Việt khi nguồn không trả slug), `normalizeInstant` (chuẩn hoá thời gian ISO-8601), `parseInt`, `splitCsv`, `isPlaceholder` (loại các giá trị "đang cập nhật"). Đừng viết lại các hàm này.

Ngay khi lớp có `@Component`, `ProviderRegistry` sẽ tự nạp — **không cần đăng ký thủ công**.

Bước 6 — Thêm retry cho lỗi mạng chớp nhoáng

Nguồn nằm sau Cloudflare (như VSMOV) thì thoảng bị ngắt ngang giữa chừng ngay sau khi container khởi động lại. Bọc lời gọi trong một vòng thử lại có backoff tăng dần, theo đúng khuôn mẫu của `VsmovProvider` (và `JikanClient`):


```
java
```

```
// Nguồn nam sau Cloudflare nen thi thoang bi ngat ngang ("Request cancelled")
// ngay sau khi container khoi dong lai. Thu lai vai lan de nuot cac cu chop nhoang nay.
private static final int NETWORK_ATTEMPTS = 3;
private static final long NETWORK_BACKOFF_MS = 200;

private <T> T call(Function<UriBuilder, URI> uriFunction, Class<T> responseType) {
    UpstreamException last = null;
    for (int attempt = 1; attempt <= NETWORK_ATTEMPTS; attempt++) {
        try {
            return client.get()
                .uri(uriFunction::apply)
                .retrieve()
                // 4xx kem body JSON: doc body de xu ly nhu du lieu rong thay vi nem loi.
                .onStatus(HttpStatus::is4xxClientError, (request, response) -> {
                })
                .body(responseType);
        } catch (RestClientException ex) {
            last = UpstreamException.network(PROVIDER_CODE, "OPhim", ex);
            log.debug("OPhim loi mang lan {} / {}: {}", attempt, NETWORK_ATTEMPTS,
                ex.getMessage());
            if (attempt == NETWORK_ATTEMPTS || !pause(attempt)) {
                log.warn("Goi OPhim that bai sau {} lan: {}", attempt, ex.getMessage());
                throw last;
            }
        }
    }
    throw last;
}

/** Nghi giua hai lan thu, dan dan lau hon. Tra false neu luong bi ngat de dung thu ngay. */
private boolean pause(int attempt) {
    try {
        Thread.sleep(NETWORK_BACKOFF_MS * attempt);
        return true;
    } catch (InterruptedException interrupted) {
        Thread.currentThread().interrupt();
        return false;
    }
}
```

```
java (tiep)
```

```
}
```

Nguyên tắc thử lại — đừng thử lại mù quáng:

Loại lỗi	Thử lại?	Vì sao
Lỗi mạng (<code>RestClientException</code> , kết nối rớt)	Có	Cú chớp nhoáng, làm lại thường ăn.
5xx (máy chủ nguồn quá tải)	Có (như <code>JikanClient</code>)	Endpoint nặng hay trả 504 tạm thời.
404	Không — trả <code>Optional.empty()</code> / danh sách rỗng	Đơn giản là không có dữ liệu.
429 (bị giới hạn)	Không	Thử lại chỉ càng bị siết.

Xử lý 4xx kèm body JSON bằng `onStatus(...)` để coi như dữ liệu rỗng thay vì ném lỗi, đúng như `NguonC/VSMOV` đang làm.

Bước 7 — Viết test cho provider mới

Kiểm thử tập trung vào **ảnh xạ JSON DTO**, dùng `MockRestServiceServer` để giả nguồn. Lấy fixture cắt từ phản hồi thật của nguồn để bắt được đúng những chỗ dễ sai. Bám theo `VsmovProviderTest`:

```
java
class OphimProviderTest {

    private OphimProvider providerWith(MockRestServiceServer[] holder) {
        RestClient.Builder builder = RestClient.builder().baseUrl("https://ophim.example");
        holder[0] = MockRestServiceServer.bindTo(builder).build();
        // Constructor rút gọn của ProviderProperties tu diện mặc định (gom ophim).
        ProviderProperties props = new ProviderProperties(null, null, null, null, null, null);
        return new OphimProvider(builder.build(), props);
    }

    @Test
    @DisplayName("Anh tra ve {} (object rỗng): map thành null, không vớ")
    void toleratesEmptyObjectPoster() {
        MockRestServiceServer[] server = new MockRestServiceServer[1];
        OphimProvider provider = providerWith(server);

        String json = """
            {"items":[
              {"_id":1,"name":"A","slug":"a","poster_url":{},
               "thumb_url":"https://ophim.example/t.jpg","year":2026}],
             "pagination":{"totalItems":1,"totalItemsPerPage":24,"currentPage":1,"totalPages":1}}
            """;
        server[0].expect(requestTo(org.hamcrest.Matchers.startsWith(
            "https://ophim.example/api/danh-sach/phim-moi-cap-nhat")))
            .andRespond(withSuccess(json, MediaType.APPLICATION_JSON));

        PageResponse<MovieSummary> page = provider.latest(MovieQuery.of(1, 24));

        assertThat(page.items()).hasSize(1);
        assertThat(page.items().get(0).posterUrl()).isNull();
        assertThat(page.items().get(0).thumbUrl()).isEqualTo("https://ophim.example/t.jpg");
    }
}
```

Bộ test tối thiểu nên bao gồm:

- `type()` và `code()` trả đúng mã.
- **Ảnh {} null, không vớ** (bài học `VSMOV` — bắt buộc có).
- `latest/search` map đúng danh sách và phân trang, gán đúng tham số `keyword/page`.
- `findBySlug` parse chi tiết, dựng đúng server/tập.
- `findBySlug` gặp 404 trả `Optional.empty()`, không ném lỗi.
- `categories()/countries()` đọc đúng danh mục.

Chạy test:

bash

```
cd backend
./mvnw test -Dtest=OphimProviderTest
```

Kết luận

Thêm một nguồn phim ở mức mã nguồn là chuỗi bảy bước có trật tự. Danh sách kiểm tra nhanh:

- [] **Bước 1** — Thêm hằng vào `ProviderType` (kèm mã URL) và cập nhật thông báo lỗi trong `from(...)`.
- [] **Bước 2** — Thêm `Endpoint` vào `ProviderProperties` (kèm mặc định) và khai báo trong `application.yml`.
- [] **Bước 3** — Viết record model Jackson `@JsonIgnoreProperties(ignoreUnknown = true)`; trường ảnh có nguy cơ để `Object`, không dùng `String`.
- [] **Bước 4** — Tạo `RestClient` bean trong `RestClientConfig`; nhớ Spring 7 `HttpHeaders` không còn là `Map`.
- [] **Bước 5** — Cài đặt `MovieProvider` (`@Component`), map sang `MovieSummary/MovieDetail`; ép ảnh qua `asUrl(...)`.
- [] **Bước 6** — Bọc lời gọi trong vòng retry backoff cho lỗi mạng / 5xx; **không** thử lại 404/429.
- [] **Bước 7** — Viết test `MockRestServiceServer`, bắt buộc có cả ảnh {} và cả 404.
- [] Cập nhật constructor `ProviderProperties(null, ...)` trong các test hiện có cho khớp số tham số mới.

Nhờ `ProviderRegistry` tự phát hiện bean, **không có bảng đăng ký thủ công** nào phải sửa — đánh dấu `@Component` là đủ.

Tài liệu và mã nguồn liên quan trong bộ:

- Mẫu tham chiếu: `backend/src/main/java/com/rapphim/warehouse/provider/vsmov/VsmovProvider.java` và `provider/nguonc/NguonCProvider.java`.
- Khuôn retry nâng cao (có xử lý 5xx/429): `provider/jikan/JikanClient.java`.
- Hàm tiện ích dùng chung: `provider/ProviderSupport.java`.
- Sổ tay xử lý sự cố DevOps khi triển khai: `../ebook/van-khan-deploy.md`.

Tài liệu API

Tài liệu: Tài liệu API
Phiên bản: 1.0
Cập nhật: 2026-09-11
Đối tượng đọc: Người tích hợp API, lập trình viên frontend
Trạng thái: Chính thức
Tag: api, rest, openapi, swagger, backend, spring-boot

Giới thiệu

RapPhim WareHouse phôi ra một REST API đóng vai trò lớp trung gian: backend Spring Boot gọi nhiều nguồn phim khác nhau (KKPhim, NguonC, VSMOV, kho riêng trên homelab), bổ sung metadata từ TheMovieDB và AniList/Jikan, rồi **chuẩn hoá tất cả về một schema DTO duy nhất** trước khi trả cho frontend. Nhờ vậy phía giao diện chỉ phải làm việc với một định dạng nhất quán, không cần biết mỗi nguồn trả JSON ra sao.

Tài liệu này mô tả bề mặt API mà chúng ta cần khi tích hợp: quy ước chung, vỏ bọc phản hồi, danh sách endpoint theo từng nhóm, bảng mã lỗi và các ví dụ gọi thật. Toàn bộ nội dung được kiểm chứng trực tiếp từ mã nguồn trong `backend/src/main/java/com/rapphim/warehouse/`, nên khớp với hiện thực.

Tài liệu API còn được **sinh tự động** bằng springdoc-openapi từ chính annotation trong code. Bản sinh động luôn có sẵn ở Swagger UI, còn tài liệu này giải thích thêm bối cảnh và cách dùng.

Các thành phần chính

Thành phần	Vai trò
<code>ApiResponse<T></code>	Vỏ bọc chuẩn cho mọi phản hồi thành công (<code>success</code> , <code>message</code> , <code>data</code> , <code>timestamp</code>).
<code>ApiError</code>	Vỏ bọc chuẩn cho mọi phản hồi lỗi (<code>4xx/5xx</code>), có trường <code>code</code> để client rẽ nhánh xử lý.
<code>PageResponse<T></code>	Kết quả dạng danh sách, gồm <code>items</code> , <code>meta</code> , <code>provider</code> .
<code>PageMeta</code>	Thông tin phân trang: <code>page</code> , <code>limit</code> , <code>totalItems</code> , <code>totalPages</code> .
<code>GlobalExceptionHandler</code>	Gom mọi ngoại lệ của tầng web về một định dạng <code>ApiError</code> duy nhất.
Tham số <code>provider</code>	Chọn nguồn dữ liệu cho mỗi lời gọi (mặc định <code>kkphim</code>).

Các nhóm endpoint (theo controller) gồm: **Movies**, **Anime**, **Homelab**, **Taxonomies**, **TMDB & NFO**, **Quản trị** và **System**.

Địa chỉ gốc và quy ước chung

Mọi endpoint đều nằm dưới base path `/api/v1`.

Môi trường	Địa chỉ gốc
Backend local (nội bộ)	<code>http://localhost:8080/api/v1</code>
Homelab (cổng công bố)	<code>http://<homelab>:7101/api/v1</code>

Các quy ước áp dụng cho toàn bộ API:

- **Định dạng:** request và response đều dùng JSON (`Content-Type: application/json`). Riêng hai endpoint xuất file trả về kiểu khác: NFO trả `application/xml`, phụ đề trả `text/vtt`.
- **Chọn nguồn:** hầu hết endpoint đọc nhận query param `provider`. Mặc định là `kkphim`; các mã hợp lệ dựng sẵn là `kkphim`, `nguồnc`, `vsmov`, `homelab`. Danh sách nguồn đang bật thật sự lấy qua `GET /api/v1/providers`.
- **Phân trang:** page bắt đầu từ 1. Với endpoint `phim`, `limit` mặc định 24 và tối đa 64; với endpoint `anime` và `TMDB`, cỡ trang là `perPage` (tối đa 50) hoặc cố định 20 (`TMDB discover`).
- **CORS:** chỉ các origin trong `rapphim.cors.allowed-origins` mới gọi được từ trình duyệt; các method cho phép là `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `OPTIONS`.
- **User-Agent (chiều ra ngoài):** khi backend gọi các nguồn bên ngoài, nó tự gán header

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) RapPhimWareHouse/1.0. Client gọi vào backend không cần header đặc biệt nào cho các endpoint đọc; riêng nhóm quản trị cần header `X-Admin-Token`.

Vỏ bọc phản hồi

ApiResponse — phản hồi thành công

Mọi phản hồi thành công đều được bọc trong `ApiResponse`. Trường `data` chứa dữ liệu nghiệp vụ thật sự (một object, một danh sách, hoặc một `PageResponse`).

```
json
{
  "success": true,
  "message": "OK",
  "data": { },
  "timestamp": "2026-09-08T16:20:00Z"
}
```

Trường	Kiểu	Ý nghĩa
<code>success</code>	boolean	Luôn <code>true</code> với phản hồi thành công.
<code>message</code>	string	Thông điệp mô tả, thường là <code>"OK"</code> .
<code>data</code>	T	Dữ liệu nghiệp vụ trả về.
<code>timestamp</code>	string (Instant, UTC)	Thời điểm server tạo phản hồi.

PageResponse và *PageMeta* — kết quả có phân trang

Các endpoint danh sách gói dữ liệu vào `PageResponse` rồi mới đặt vào `data` của `ApiResponse`.

json

```
{
  "items": [ ],
  "meta": { "page": 1, "limit": 24, "totalItems": 8993, "totalPages": 375 },
  "provider": "kkphim"
}
```

Trường	Kiểu	Ý nghĩa
items	mảng T	Danh sách phần tử của trang hiện tại.
meta.page	int	Trang hiện tại, bắt đầu từ 1.
meta.limit	int	Số phần tử tối đa trên một trang.
meta.totalItems	long	Tổng số phần tử tìm được.
meta.totalPages	int	Tổng số trang.
provider	string	Nguồn dữ liệu đã phục vụ request này.

ApiError — phản hồi lỗi

Khi thất bại, API trả về `ApiError` với `success = false` và trường `code` để client xử lý theo từng trường hợp thay vì đọc chuỗi `message`.

json

```
{
  "success": false,
  "status": 404,
  "code": "MOVIE_NOT_FOUND",
  "message": "Không tìm thấy phim với slug 'abc'",
  "path": "/api/v1/movies/abc",
  "details": [ ],
  "timestamp": "2026-09-08T16:20:00Z"
}
```

Trường	Kiểu	Ý nghĩa
success	boolean	Luôn <code>false</code> .
status	int	Mã HTTP status.
code	string	Mã lỗi nội bộ, dùng để client rẽ nhánh.
message	string	Mô tả lỗi cho người dùng.
path	string	Đường dẫn đã gọi.
details	mảng string	Chi tiết lỗi (ví dụ lỗi validate từng field), rỗng nếu không có.
timestamp	string (Instant, UTC)	Thời điểm xảy ra lỗi.

Endpoint nhóm Movies

Base: `/api/v1/movies`. Trừ khi ghi rõ, tham số chung là `page` (mặc định 1, tối thiểu 1), `limit` (mặc định 24, tối đa 64) và `provider` (mặc định `kkphim`).

Met hod	Đường dẫn	Mô tả	Tham số chính
GET	<code>/latest</code>	Phim mới cập nhật, dùng cho trang chủ.	<code>page</code> , <code>limit</code> , <code>provider</code>
GET	<code>/</code>	Danh sách theo nhóm, kèm bộ lọc và sắp xếp.	<code>type</code> , <code>category</code> , <code>country</code> , <code>year</code> , <code>sortField</code> , <code>sortType</code> , <code>page</code> , <code>limit</code> , <code>provider</code>
GET	<code>/search</code>	Tìm theo tên tiếng Việt hoặc tên gốc.	<code>keyword</code> (bắt buộc), <code>page</code> , <code>limit</code> , <code>provider</code>
GET	<code>/category/{slug}</code>	Phim theo thể loại.	<code>slug</code> (path), <code>country</code> , <code>year</code> , <code>page</code> , <code>limit</code> , <code>provider</code>
GET	<code>/country/{slug}</code>	Phim theo quốc gia.	<code>slug</code> (path), <code>category</code> , <code>year</code> , <code>page</code> , <code>limit</code> , <code>provider</code>
GET	<code>/year/{year}</code>	Phim theo năm phát hành.	<code>year</code> (path), <code>category</code> , <code>country</code> , <code>page</code> , <code>limit</code> , <code>provider</code>
GET	<code>/batch</code>	Trạng thái hiện tại của nhiều phim để dò tập mới.	<code>slugs</code> (bắt buộc, tối đa 30), <code>provider</code>
GET	<code>/slug</code>	Chi tiết phim kèm danh sách server và tập.	<code>slug</code> (path), <code>provider</code>

Giá trị hợp lệ của `type` (enum `ListType`): `phim-bo`, `phim-le`, `tv-shows`, `hoat-hinh`, `phim-vietsub`, `phim-thuyet-minh`, `phim-long-tieng`, `dang-chieu`. Với endpoint `/`, tham số `sortField` nhận `modified.time` | `_id` | `year`, còn `sortType` nhận `asc` | `desc`.

Lưu ý: nguồn `nguonc` chỉ hỗ trợ `phim-bo`, `phim-le`, `tv-shows`, `hoat-hinh` và bỏ qua các bộ lọc. Endpoint `/batch` bỏ qua các phim không còn tồn tại thay vì làm hỏng cả danh sách; client giữ bản chụp cũ rồi so `episodeCurrent` và `modifiedAt` để biết có tập mới.

Endpoint nhóm Anime

Base: `/api/v1`. Metadata anime lấy từ AniList (chính) và Jikan/MyAnimeList (dự phòng); danh sách và tìm kiếm tự động chuyển nguồn khi AniList lỗi — kiểm tra trường `source` để biết id thuộc nguồn nào.

Met hod	Đường dẫn	Mô tả	Tham số chính
GET	<code>/anime/trending</code>	Anime đang được quan tâm nhất.	<code>page</code> (1–500), <code>perPage</code> (1–50)
GET	<code>/anime/search</code>	Tìm anime theo từ khoá.	<code>keyword</code> (bắt buộc), <code>page</code> (1–500), <code>perPage</code> (1–50)
GET	<code>/anime/{id}</code>	Chi tiết một anime theo mã.	<code>id</code> (path, int), <code>source</code> (anilist jikan)

Met hod	Đường dẫn	Mô tả	Tham số chính
GET	/anime/gen res	Danh mục thể loại anime để điền bộ lọc.	—

Quan trọng: khi gọi chi tiết /anime/{id} phải kèm đúng source của mã đó (lấy từ trường source trong kết quả danh sách), vì AniList và MyAnimeList có không gian mã khác nhau. Chi tiết không dự phòng sang nguồn khác.

Endpoint nhóm Homelab

Base: /api/v1/homelab. Nhóm này phục vụ kho phim riêng chạy trên homelab (đọc qua ZCloud).

Meth od	Đường dẫn	Mô tả	Tham số chính
GET	/subtitle	Tải phụ đề của kho riêng, đã chuyển sang WebVTT.	key (bắt buộc)

Endpoint trả Content-Type: text/vtt;charset=UTF-8 (không bọc trong ApiResponse) và đặt Cache-Control giữ lại 6 giờ. Lý do phải đi qua backend: thẻ <track> của trình duyệt chỉ đọc WebVTT trong khi phụ đề gốc thường là .srt/.ass, và kho đòi xác thực mà trình duyệt không được giữ khoá.

Endpoint nhóm Taxonomies

Base: /api/v1. Các danh mục dùng để dựng thanh lọc của giao diện.

Method	Đường dẫn	Mô tả	Tham số chính
GET	/categories	Toàn bộ thể loại.	provider
GET	/countries	Toàn bộ quốc gia.	provider

slug trả về ở đây dùng làm tham số cho /movies/category/{slug}, /movies/country/{slug} hoặc các bộ lọc category / country.

Endpoint nhóm TMDB & NFO

Base: /api/v1. Nhóm này bổ sung metadata từ TheMovieDB và xuất file NFO cho trình quản lý thư viện phim.

Met hod	Đườn g dẫn	Mô tả	Tham số chính
GET	/movie s/{slu g}/tmd b	Metadata TMDB của một phim (đối chiếu theo mã TMDB nguồn trả kèm).	slug (path), provider
GET	/movie s/{slu g}/nfo	Xuất file NFO (XML) cho Kodi / Jellyfin / Emby.	slug (path), provider, enrich (mặc định true)

Met hod	Đườn g dẫn	Mô tả	Tham số chính
GET	/tmdb/discover	Proxy của discover/movie, gợi ý phim theo điểm/thể loại/ngôn ngữ.	page (1-500), sortBy, withGenres, withOriginalLanguage, year, voteAverageGte, voteCountGte, region, includeAdult
GET	/tmdb/movies/{id}	Chi tiết một phim trên TMDB theo mã TMDB.	id (path), type (movie tv)
GET	/tmdb/person/{id}	Diễn viên kèm các phim từng đóng.	id (path)
GET	/tmdb/genres	Danh mục thể loại của TMDB.	type (movie tv)
GET	/tmdb/status	Backend đã cấu hình khoá TMDB hay chưa.	—

Các endpoint /tmdb/* cần khoá TMDB. Khi chưa cấu hình, chúng trả 503 TMDB_NOT_CONFIGURED, phần còn lại của API không bị ảnh hưởng. Riêng /movies/{slug}/nfo **chạy được ngay không cần khoá** vì KKPhim đã trả sẵn mã TMDB/IMDb; tham số enrich=false sẽ bỏ qua bước bổ sung TMDB. Với /tmdb/discover, TMDB cố định 20 kết quả mỗi trang.

Endpoint nhóm Quản trị

Base: /api/v1/admin. Cho phép thêm và sửa nguồn phim lúc đang chạy mà không phải sửa mã nguồn. Các endpoint **đọc** thì ai cũng gọi được; các endpoint **sửa đổi** đòi khoá gửi qua header X-Admin-Token.

Met hod	Đường dẫn	Mô tả	Tham số chính
GET	/audit	Nhật ký các thay đổi gần đây (50 mục).	Header X-Admin-Token
GET	/settings	Tình trạng các khoá (không kèm giá trị).	—
POST	/settings	Đặt hoặc xoá một khoá.	Header X-Admin-Token, body { "name", "value" }
GET	/sources	Danh sách nguồn tự thêm.	—
POST	/sources	Thêm hoặc cập nhật một nguồn.	Header X-Admin-Token, body CustomSource
POST	/sources/probe	Gọi thử một nguồn trước khi lưu.	Header X-Admin-Token, body CustomSource
DELETE	/sources/{id}	Xoá một nguồn.	Header X-Admin-Token, id (path)

Khi chưa đặt biến môi trường RAPPIM_ADMIN_TOKEN, phần sửa đổi tự tắt hẳn và trả 503 ADMIN_NOT_CONFIGURED — để mở cho ai cũng đổi được nguồn phim là mời kẻ khác trở hệ thống sang địa chỉ bất kỳ. Khoá sai trả 401 UNAUTHORIZED. Nhật ký và tình trạng khoá cố ý **không** trả về giá trị khoá thật.

Endpoint nhóm System

Base: `/api/v1`. Các endpoint mô tả chính hệ thống, hữu ích khi frontend dựng menu động.

Met hod	Đường dẫn	Mô tả	Tham số chính
GET	<code>/status</code>	Tình trạng cấu hình của hệ thống (không trả về khoá nào).	—
GET	<code>/providers</code>	Các mã nguồn có thể truyền vào tham số <code>provider</code> .	—
GET	<code>/list-types</code>	Các nhóm danh sách kèm nhãn hiển thị.	—

Bảng mã lỗi

Mọi lỗi của tầng web được `GlobalExceptionHandler` gom về một `ApiError`. Bảng dưới liệt kê đầy đủ các mã lỗi và HTTP status tương ứng.

	HTT P	Ý nghĩa
INVALID_PARAMETER	400	Tham số sai định dạng hoặc ngoài miền giá trị (ví dụ <code>type</code> không hợp lệ).
MISSING_PARAMETER	400	Thiếu tham số bắt buộc (ví dụ thiếu <code>keyword</code>).
VALIDATION_FAILED	400	Không qua bước validate (ví dụ <code>limit > 64</code> , <code>page < 1</code>).
UNAUTHORIZED	401	Khoá quản trị <code>X-Admin-Token</code> không đúng.
MOVIE_NOT_FOUND	404	Không có phim với slug đã cho.
ANIME_NOT_FOUND	404	Nguồn không có anime với mã (và <code>source</code>) đã cho.
TMDB_NOT_FOUND	404	Không có bản ghi tương ứng trên TheMovieDB.
SUBTITLE_NOT_FOUND	404	Không tìm thấy phụ đề với <code>key</code> đã cho trong kho homelab.
SOURCE_NOT_FOUND	404	Không có nguồn tự thêm với <code>id</code> đã cho.
ENDPOINT_NOT_FOUND	404	Sai đường dẫn endpoint.
UPSTREAM_ERROR	502	Nguồn phim bên ngoài lỗi hoặc timeout.
UPSTREAM_BLOCKED	502	Kết nối tới nguồn bị cắt/chặn theo tên miền (thường do DNS hoặc TLS bị chặn).
TMDB_NOT_CONFIGURED	503	Chưa cấu hình khoá TheMovieDB.
ADMIN_NOT_CONFIGURED	503	Chưa đặt <code>RAPPIM_ADMIN_TOKEN</code> nên không sửa được nguồn.
INTERNAL_ERROR	500	Lỗi không xác định phía server.

Ví dụ request và response

Lấy phim mới cập nhật

Gọi endpoint trang chủ với 2 phim mỗi trang.

http

```
GET /api/v1/movies/latest?page=1&limit=2&provider=kkphim HTTP/1.1
Host: localhost:8080
Accept: application/json
```

Phản hồi là một `PageResponse<MovieSummary>` nằm trong `data`:

json

```
{
  "success": true,
  "message": "OK",
  "data": {
    "items": [
      {
        "id": "b107d3b6dba125cdb0732029601eeee9",
        "slug": "doi-chung",
        "name": "Doi Chung",
        "originName": "Cause Of Death",
        "posterUrl": "https://phimimg.com/uploads/movies/doi-chung-poster.webp",
        "thumbUrl": "https://phimimg.com/uploads/movies/doi-chung-thumb.webp",
        "year": 2026,
        "type": "series",
        "quality": "FHD",
        "lang": "Long Tieng",
        "time": "45 phut/tap",
        "episodeCurrent": "Tap 2",
        "categories": [
          { "id": "9822be11d2ccc29c7172c78b8af8ff5", "name": "Hanh Dong", "slug": "hanh-dong" }
        ]
      }
    ],
    "countries": [
      { "id": "3e64d0e0d9c1d2f7e0e3f4a5b6c7d8e9", "name": "Han Quoc", "slug": "han-quoc" }
    ],
    "tmdb": null,
    "imdb": null,
    "provider": "kkphim",
    "modifiedAt": "2026-09-08T20:42:25Z"
  },
  "meta": { "page": 1, "limit": 2, "totalItems": 8993, "totalPages": 4497 },
  "provider": "kkphim",
  "timestamp": "2026-09-08T16:20:00Z"
}
```

Lấy chi tiết một phim

http

```
GET /api/v1/movies/doi-chung?provider=kkphim HTTP/1.1
Host: localhost:8080
Accept: application/json
```

Phản hồi là một `MovieDetail` (rút gọn phần `servers` để dễ đọc):

json

```
{
  "success": true,
  "message": "OK",
  "data": {
    "id": "b107d3b6dba125cdb0732029601eeee9",
    "slug": "doi-chung",
    "name": "Doi Chung",
    "originName": "Cause Of Death",
    "content": "Noi dung phim...",
    "year": 2026,
    "type": "series",
    "status": "ongoing",
    "quality": "FHD",
    "lang": "Long Tieng",
    "time": "45 phut/tap",
    "episodeCurrent": "Tap 2",
    "episodeTotal": "25",
    "actors": [],
    "directors": [],
    "categories": [
      { "id": "9822be111d2ccc29c7172c78b8af8ff5", "name": "Hanh Dong", "slug": "hanh-dong" }
    ],
    "countries": [],
    "servers": [],
    "tmdb": null,
    "imdb": null,
    "provider": "kkphim",
    "modifiedAt": "2026-09-08T20:42:25Z"
  },
  "timestamp": "2026-09-08T16:20:00Z"
}
```

Tìm kiếm bằng dòng lệnh

Ba lời gọi mẫu bằng `curl` (nhớ mã hoá URL cho khoảng trắng trong `keyword`):

bash

```
curl "http://localhost:8080/api/v1/movies/latest?page=1&limit=12"
curl "http://localhost:8080/api/v1/movies?type=phim-bo&country=han-quoc&year=2026"
curl "http://localhost:8080/api/v1/movies/search?keyword=nguoi%20nhen&provider=nguồnc"
```

Một phản hồi lỗi

Gọi chi tiết với slug không tồn tại sẽ nhận 404 kèm `code = MOVIE_NOT_FOUND`:

http

```
GET /api/v1/movies/khong-co-that HTTP/1.1
Host: localhost:8080
Accept: application/json
```

json

```
{
  "success": false,
  "status": 404,
  "code": "MOVIE_NOT_FOUND",
  "message": "Không tìm thấy phim với slug 'khong-co-that'",
  "path": "/api/v1/movies/khong-co-that",
  "details": [],
  "timestamp": "2026-09-08T16:20:00Z"
}
```

Thêm một nguồn qua endpoint quản trị

Endpoint sửa đổi đòi header X-Admin-Token:

http

```
POST /api/v1/admin/sources HTTP/1.1
Host: localhost:8080
Content-Type: application/json
X-Admin-Token: <RAPPIM_ADMIN_TOKEN>
```

```
{
  "id": "my-source",
  "name": "Ngon rieng",
  "baseUrl": "https://vi-du.local",
  "enabled": true
}
```

Nếu chưa đặt RAPPIM_ADMIN_TOKEN, phản hồi là 503 với code = ADMIN_NOT_CONFIGURED; nếu khoá sai là 401 với code = UNAUTHORIZED.

Swagger UI và OpenAPI JSON

Tài liệu API được sinh tự động từ annotation trong code (springdoc-openapi), nên không bao giờ lệch với hiện thực. Swagger UI còn cho phép gọi thử trực tiếp từng endpoint qua nút *Try it out*.

Tài nguyên	Địa chỉ (local)
Swagger UI	<code>http://localhost:8080/swagger-ui.html</code>
OpenAPI JSON (động)	<code>http://localhost:8080/v3/api-docs</code>
Bản xuất tĩnh trong repo	<code>../openapi.json</code>

Trên homelab, thay localhost:8080 bằng <homelab>:7101. Nếu chỉ cần kiểm tra nhanh backend đã có những khoá và nguồn nào mà không lộ giá trị, gọi GET /api/v1/status:

bash

```
curl "http://localhost:8080/api/v1/status"
```

Phản hồi có dạng dưới đây (các giá trị boolean phản ánh cấu hình hiện tại của máy):

```
json
{
  "success": true,
  "message": "OK",
  "data": {
    "providers": ["kkphim", "nguồn"],
    "builtInCount": 4,
    "customCount": 0,
    "customEnabledCount": 0,
    "tmdbConfigured": false,
    "tmdbKeyShape": "none",
    "homelabConfigured": false,
    "adminConfigured": false
  },
  "timestamp": "2026-09-08T16:20:00Z"
}
```

Kết luận

REST API của RapPhim Warehouse gói mọi phản hồi vào `ApiResponse` (thành công) hoặc `ApiError` (thất bại), phân trang qua `PageResponse/PageMeta`, và chọn nguồn qua tham số `provider`. Khi tích hợp, hãy luôn đọc trường `code` của lỗi để rẽ nhánh xử lý thay vì dựa vào chuỗi `message`, và tôn trọng ràng buộc `limit` tối đa 64 cùng `page` bắt đầu từ 1.

Khi cần tra cứu chi tiết một schema hoặc thử một endpoint, hãy mở **Swagger UI** — đó là nguồn sự thật cập nhật theo code. Bản OpenAPI tĩnh nằm ở `../openapi.json` tiện cho việc sinh client hoặc diff giữa các phiên bản.

Hướng dẫn triển khai bằng Docker (homelab)

Tài liệu: Triển khai RapPhim WareHouse bằng Docker trên homelab
Phiên bản: 1.0
Cập nhật: 2026-09-11
Đối tượng đọc: DevOps / người vận hành homelab
Trạng thái: Chính thức
Tag: docker, docker-compose, homelab, coredns, deploy, ssh

Giới thiệu

Tài liệu này hướng dẫn triển khai toàn bộ RapPhim WareHouse lên một máy chủ homelab bằng Docker Compose. Sau khi làm theo, chúng ta sẽ có ba container chạy song song: một bộ phân giải tên miền riêng (CoreDNS), một backend Spring Boot, và một frontend Next.js, tất cả nằm trong cùng một mạng Docker riêng.

Tại sao lại cần một quy trình riêng cho homelab thay vì chỉ `docker compose up`? Có hai lý do thực tế mà repo đã gặp phải và đã xử lý sẵn:

- **Xung đột cổng.** Cổng 8080 trên máy homelab đã có dịch vụ khác (ZCloud) dùng, nên chúng ta đưa backend ra ngoài ở cổng 7101 và web ở cổng 7100. Bên trong mạng Docker, hai dịch vụ vẫn nghe 8080 và 3000 như bình thường.
- **DNS của ISP bị can thiệp.** Trên mạng của máy này, một số tên miền (ví dụ `api.themoviedb.org`) bị chèn phản hồi DNS giả trả về 127.0.0.1. Lỗi hiện ra là "I/O error", trông như backend hỏng, nhưng thực chất là không bao giờ kết nối tới nơi. Vì vậy chúng ta chạy một bộ phân giải CoreDNS dùng DNS-over-TLS để đi trong đường mã hoá.

Máy đích không cài `rsync`, nên việc đưa mã nguồn lên được thực hiện qua `tar` truyền qua SSH bằng script `deploy.sh`. Tài liệu sẽ đi qua cả hai cách: chạy tại chỗ và triển khai từ xa.

Các thành phần chính

Toàn bộ ngăn xếp được mô tả trong `docker-compose.yml` (bản build từ mã nguồn) và `docker-compose.prod.yml` (bản chạy từ ảnh dựng sẵn).

Dịch vụ	Ảnh / build	Container	Cổng ra ngoài	Cổng nội bộ	Vai trò
dns	coredns/coredns:1.14.7	rapphim-dns	(không)	53	Bộ phân giải DNS-over-TLS riêng, IP tĩnh 172.29.0.53
backend	build ./backend	rapphim-backend	7101	8080	REST API tổng hợp, chuẩn hoá dữ liệu, cache Caffeine
web	build ./frontend	rapphim-web	7100	3000	Giao diện Next.js, gọi backend qua route handler nội bộ

Các đặc điểm mạng đáng lưu ý:

Thuộc tính	Giá trị	Vì sao
Mạng	rapphim (driver bridge)	Mạng riêng cho ba container

Thuộc tính	Giá trị	Vì sao
Dải mạng	172.29.0.0/16	Các dải 172.17-172.23 trên máy đã có dự án khác dùng
IP cố định của DNS	172.29.0.53	Backend cần một địa chỉ tĩnh để trỏ DNS
Volume dữ liệu	rapphim-data gắn vào /app/data	Giữ nguồn tự thêm qua các lần dựng lại container

Backend còn được khai báo `extra_hosts` là `host.docker.internal:host-gateway` để gọi ngược về ZCloud đang chạy trên chính máy chủ (không nằm trong Docker).

Chuẩn bị tệp .env

Mọi giá trị thay đổi theo máy đều nằm trong một tệp `.env` đặt cạnh `docker-compose.yml`. Chép từ mẫu rồi điền:

```
bash
cp .env.example .env
```

Các biến trong `.env.example` và ý nghĩa:

Biến	Mặc định	Mô tả
WEB_PORT	7100	Cổng web đưa ra ngoài
BACKEND_PORT	7101	Cổng backend đưa ra ngoài
PUBLIC_WEB_URL	http://192.168.100.169:7100	Địa chỉ web nhìn từ trình duyệt; chỉ dùng để mở CORS cho ai gọi thẳng API
ZCLOUD_API_KEY	(trống)	Khoá gọi kho phim riêng ZCloud, gửi qua header <code>x-api-key</code> (ưu tiên)
ZCLOUD_PASSWORD	(trống)	Mật khẩu ZCloud, dùng khi chưa tạo khoá
ZCLOUD_PREFIX	(trống)	Tiền tố đường dẫn trong kho ZCloud
ZCLOUD_BASE_URL	http://host.docker.internal:8080	Địa chỉ ZCloud; nó chạy trên máy chủ, không phải trong Docker
ADMIN_TOKEN	(trống)	Khoá để thêm/sửa nguồn phim tại <code>/quan-tri</code> ; để trống thì trang đó chỉ xem được
TMDB_ACCESS_TOKEN	(trống)	Token đọc TMDB v4 (ưu tiên); thiếu thì các endpoint TMDB trả 503
TMDB_API_KEY	(trống)	Khoá API TMDB v3
BACKEND_MEMORY_LIMIT	1g	Giới hạn RAM backend (chỉ dùng ở bản prod)
JAVA_OPTS	-Xmx640m	Tuỳ chọn JVM (chỉ dùng ở bản prod)

Biến	Mặc định	Mô tả
WEB_MEM_LIMIT	512m	Giới hạn RAM web (chỉ dùng ở bản prod)

Lưu ý an toàn: đừng đặt khoá ZCloud hay ADMIN_TOKEN vào tệp có ý định chia sẻ. ADMIN_TOKEN nên là một chuỗi dài và ngẫu nhiên, vì ai có nó là đổi được nơi lấy phím của cả hệ thống. Nếu để trống các khoá ZCloud thì nguồn homelab tự tắt, phần còn lại của hệ thống vẫn chạy.

Build và chạy tại chỗ

Khi mã nguồn đã có sẵn trên máy đích (ví dụ sau khi `deploy.sh` gửi lên), chúng ta build ảnh và khởi động bằng một lệnh:

```
bash
docker compose up -d --build
```

Lệnh này đọc `docker-compose.yml`, build ảnh `rapphim-backend:latest` từ `backend/Dockerfile` và `rapphim-web:latest` từ `frontend/Dockerfile`, kéo ảnh CoreDNS, rồi khởi động cả ba dịch vụ ở chế độ nền. Lần đầu sẽ mất vài phút vì phải tải phụ thuộc Maven và npm.

Backend dùng ảnh nhiều tầng: tầng build là `maven:3.9-eclipse-temurin-17`, tầng chạy là `bellsoft/liberica-openjre-alpine:17` (giữ nền Alpine vì `busybox` có sẵn `wget` cho HEALTHCHECK, và bản này có cả `arm64`). Frontend build theo dạng `standalone` của Next.js rồi chạy bằng `node server.js`.

Kiểm tra các container đang chạy:

```
bash
docker compose ps
```

Kết quả (tên container là cố định theo cấu hình):

NAME	IMAGE	STATUS
rapphim-dns	coredns/coredns:1.14.7	Up
rapphim-backend	rapphim-backend:latest	Up
rapphim-web	rapphim-web:latest	Up

Bản chạy từ ảnh dựng sẵn (prod override)

`docker-compose.prod.yml` là một tệp compose độc lập (không phải file phủ chồng lên bản gốc), dùng khi không muốn build tại chỗ. Khác biệt chính so với `docker-compose.yml`:

- Backend và web dùng ảnh có sẵn `ghcr.io/nguyenquocanhz/rapphim-backend:${TAG:-latest}` và `ghcr.io/nguyenquocanhz/rapphim-web:${TAG:-latest}` thay vì build.
- Mỗi container có `mem_limit`: backend 1g, web 512m, dns 64m. Đặt `mem_limit` cho backend vừa ghim trần heap JVM (ảnh đặt `MaxRAMPercentage=75` sẽ đọc RAM cả host nếu không giới hạn), vừa để OOM-killer của kernel chỉ giết riêng container này thay vì chọn nạn nhân toàn máy.

Cách chạy bản này (không cần mã nguồn):

```
bash
```

```
curl -O https://raw.githubusercontent.com/nguyenquocanhz/RapPhimWareHouse/main/docker-compose.prod.yml
curl -o .env https://raw.githubusercontent.com/nguyenquocanhz/RapPhimWareHouse/main/.env.example
docker compose -f docker-compose.prod.yml up -d
```

Ảnh không nhúng địa chỉ nào bên trong nên chạy được ở bất kỳ máy nào; đổi cổng thì chỉ cần sửa `WEB_PORT` / `BACKEND_PORT` trong `.env`.

Triển khai từ xa bằng `deploy.sh`

Trên máy phát triển, `deploy.sh` đóng gói mã nguồn, gửi qua SSH tới máy homelab rồi dựng lại bằng Docker Compose. Máy đích không cần cài `rsync`; chỉ cần có `tar` ở cả hai đầu.

Chạy toàn bộ quy trình (gửi mã nguồn, build lại, khởi động):

```
bash
```

```
./deploy.sh
```

Các chế độ khác:

```
bash
```

```
./deploy.sh --no-build # chỉ gửi mã nguồn rồi khởi động lại, không build
./deploy.sh --logs     # xem log đang chạy rồi thoát
./deploy.sh --help     # in phân hướng dẫn ở đầu tệp
```

Các biến cấu hình của script

Có thể ghi đè bằng biến môi trường khi gọi script:

Biến	Mặc định	Ý nghĩa
<code>SSH_HOST</code>	<code>nqatech@192.168.100.169</code>	Tài khoản và địa chỉ máy đích
<code>SSH_KEY</code>	<code>\$HOME/.ssh/acer-nitro</code>	Khoá SSH dùng để đăng nhập
<code>REMOTE_DIR</code>	<code>rapphim</code>	Thư mục đích trong <code>\$HOME</code> của máy chủ

Các bước script thực hiện

- Kiểm tra kết nối.** Chạy `docker compose version` qua SSH (`BatchMode=yes`) để chắc máy đích vào được và đã có Docker Compose. Không được thì dừng và báo khoá đang dùng.
- Xoá mã nguồn cũ.** Xoá các thư mục `backend` `frontend` `docs` `dub` trên máy đích trước khi gửi bản mới. Việc này cần thiết vì giải nén `tar` chỉ ghi đè tệp đang có, không xoá tệp đã bị xoá ở bản mới — nếu không xoá trước, tệp cũ vẫn còn và vẫn build vào ảnh. Tệp `.env` của người dùng nằm ở gốc nên được giữ nguyên.
- Gửi mã nguồn.** Đóng gói bằng `tar czf` rồi bung ra ở máy đích, loại các thư mục sinh khi build và tệp riêng:

bash

```
tar czf - -C "$HERE" \
  --exclude=node_modules --exclude=target --exclude=.next --exclude=.git \
  --exclude='*.log' --exclude=.env --exclude=__pycache__ \
  --exclude=ttsenv --exclude=_media \
  backend frontend docs dub README.md \
  docker-compose.yml docker-compose.prod.yml Corefile .env.example \
  | ssh_run "mkdir -p ~/${REMOTE_DIR} && tar xzf - -C ~/${REMOTE_DIR}"
```

Đáng chú ý: `.env` bị loại khỏi gói (máy đích tự giữ tệp này, ghi đè lên là mất khoá).

1. **Tạo `.env` lần đầu.** Nếu máy đích chưa có `.env`, script chép từ `.env.example` để compose không hỏng, kèm nhắc "nhớ điền khoá ZCloud".
2. **Dựng lại.** Với chế độ mặc định chạy `docker compose up -d --build`; với `--no-build` chỉ chạy `docker compose up -d`.
3. **Chờ và kiểm tra.** Lặp tối đa 30 lần, mỗi lần cách 3 giây (tổng 90 giây), gọi `GET /actuator/health` của backend và `GET /` của web. Nếu cả hai trả về được thì báo *Xong* kèm địa chỉ web; nếu quá 90 giây vẫn chưa trả lời thì thoát với mã lỗi và gợi ý xem log.

Địa chỉ để kiểm tra được suy ra từ `SSH_HOST` và các cổng đọc từ `.env` (`WEB_PORT`, `BACKEND_PORT`), mặc định `http://<host>:7100` cho web và `http://<host>:7101` cho API.

CoreDNS: bộ phân giải tên miền riêng

Container `dns` chạy CoreDNS với cấu hình trong `Corefile`. Vai trò của nó là giải mã tên miền qua DNS-over-TLS để tránh việc DNS thường bị chèn phản hồi giả.

corefile

```
.:53 {
  forward . tls://8.8.8.8 tls://8.8.4.4 {
    tls_servername dns.google
  }
  cache 300
  errors
}
```

- `forward . tls://...:` chuyển mọi truy vấn qua kênh TLS tới Google DNS, `tls_servername dns.google` để xác thực chứng chỉ.
- `cache 300:` đệm 300 giây, đỡ phải mở kết nối TLS mới mỗi lần hỏi.

Backend được cấu hình trỏ thẳng vào bộ phân giải này bằng địa chỉ IP tĩnh. Trong `docker-compose.yml`, dịch vụ `backend` khai báo:

yaml

```
dns:
  - 172.29.0.53
depends_on:
  - dns
```

Phải ghi bằng địa chỉ IP chứ không dùng tên dịch vụ `dns`, vì Docker không nhận tên dịch vụ ở mục `dns:` của container. Đây cũng là lý do mạng `rapphim` được cố định dải `172.29.0.0/16` và gán IP tĩnh `172.29.0.53` cho container `dns` — để backend luôn tìm thấy đúng địa chỉ đó qua các lần dựng lại.

Health-check, xem log, cập nhật, rollback

Kiểm tra sức khỏe

Cả hai ảnh đều có HEALTHCHECK nội bộ: backend gọi wget tới /actuator/health, web gọi tới /. Kiểm tra thủ công từ ngoài:

```
bash
curl -fsS http://192.168.100.169:7101/actuator/health
```

Kết quả khi backend khỏe:

```
json
{"status": "UP"}
```

Cấu hình actuator (application.yml) mở các endpoint health, info, caches, metrics và đặt show-details: always, nên /actuator/health còn kèm chi tiết từng thành phần. Web trả về mã 200 ở đường dẫn gốc là đủ để coi là sẵn sàng.

Xem log

```
bash
# Tren may dat: theo doi log ca ba dich vu
docker compose logs --tail 60 -f

# Chi mot dich vu
docker compose logs -f backend

# Tu may phat trien, qua deploy.sh
./deploy.sh --logs
```

Cập nhật

- **Từ mã nguồn:** chạy lại ./deploy.sh. Script tự gửi bản mới rồi docker compose up -d --build; Docker chỉ dựng lại các tầng thay đổi.
- **Từ ảnh dựng sẵn (bản prod):** kéo ảnh mới rồi dựng lại:

```
bash
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

Dữ liệu nguồn tự thêm nằm trong volume raphim-data, không bị mất khi dựng lại container.

Rollback

Bản prod ghim phiên bản qua biến TAG. Để quay về một phiên bản cũ, đặt TAG trong .env (hoặc truyền trực tiếp) rồi dựng lại:

```
bash
TAG=v1.2.3 docker compose -f docker-compose.prod.yml up -d
```

Với bản build từ mã nguồn, cách quay lui là git checkout về commit mong muốn rồi chạy lại ./deploy.sh.

Lưu ý tài nguyên: dịch vụ thuyết minh (dub) đang tắt

Dịch vụ thuyết minh (dub) hiện đang **tắt** trong `docker-compose.yml`. Lý do là phần ML của nó (PaddleOCR ONNX + ffmpeg + TTS) quá nặng so với sức homelab hiện tại. Mã nguồn vẫn còn nguyên ở thư mục `dub/` (có sẵn `Dockerfile`, dịch vụ FastAPI nghe cổng 8080 nội bộ) và tầng giao diện cũng còn hỗ trợ.

Để bật lại, cần làm ba việc:

1. **Thêm lại service** `dub` vào `docker-compose.yml`, build từ `./dub` và nối vào mạng `rapphim`:

```
yaml
dub:
  build:
    context: ./dub
  container_name: rapphim-dub
  restart: unless-stopped
  networks:
    - rapphim
```

1. **Đặt DUB_INTERNAL_URL cho web** để route handler `frontend/src/app/api/dub/[...path]/route.ts` biết gọi tới đâu (bên trong mạng Docker là tên dịch vụ và cổng nội bộ):

```
yaml
web:
  environment:
    API_INTERNAL_URL: http://backend:8080
    DUB_INTERNAL_URL: http://dub:8080
```

1. **Bật** `cờ` `NEXT_PUBLIC_DUB_ENABLED` khi build lại ảnh web. Trong `frontend/src/components/watch/VideoPlayer.tsx`, cờ này được đọc là `process.env.NEXT_PUBLIC_DUB_ENABLED === "true"`; vì là biến `NEXT_PUBLIC_` dùng trong component phía trình duyệt, nó được nhúng lúc build, nên phải đặt `NEXT_PUBLIC_DUB_ENABLED=true` rồi build lại web.

Sau ba bước, dựng lại bằng `docker compose up -d --build`. Hãy cân nhắc kỹ tài nguyên trước khi bật: dịch vụ này là nguyên nhân ngốn RAM/CPU khiến nó bị tắt ngay từ đầu.

Kết luận

Chúng ta đã đi qua toàn bộ vòng đời triển khai RapPhim Warehouse trên homelab: kiến trúc ba container `dns` / `backend` / `web` trong mạng `172.29.0.0/16`, chuẩn bị `.env`, build và chạy bằng `docker compose up -d --build`, triển khai từ xa qua `deploy.sh`, vai trò của CoreDNS với IP tĩnh `172.29.0.53`, cùng các thao tác vận hành thường ngày (health-check, log, cập nhật, rollback) và cách bật lại dịch vụ `dub` khi có đủ tài nguyên.

Các tài liệu liên quan trong bộ:

- `.env.example` — mẫu cấu hình đầy đủ kèm chú thích.
- `docs/openapi.json` — đặc tả API để kiểm thử các endpoint sau khi triển khai.
- `docs/ebook/van-khan-deploy.md` — sổ tay sự cố DevOps đi kèm.
- `README.md` ở gốc dự án — tổng quan kiến trúc và cách chạy ở môi trường phát triển.

Tham chiếu cấu hình

Tài liệu

Phiên bản: 1.0

Cập nhật: 2026-09-11

Đối tượng đọc: DevOps, lập trình viên

Trạng thái: Chính thức

Tag: cấu hình, biến môi trường, backend, Spring Boot, Docker, TMDB, ZCloud, CORS

Giới thiệu

Backend của RapPhim WareHouse là một lớp trung gian: nó gọi nhiều nguồn phim, chuẩn hoá dữ liệu về một schema DTO duy nhất, cache lại bằng Caffeine rồi phục vụ frontend. Toàn bộ hành vi của lớp trung gian này — gọi nguồn nào, khoá dịch vụ ngoài nào, ai được phép gọi API — đều điều khiển bằng **biến môi trường** và các giá trị mặc định trong `application.yml`.

Tài liệu này là bản tham chiếu đầy đủ cho các cấu hình đó. Chúng ta liệt kê từng biến kèm giá trị mặc định, ánh xạ sang lớp `*Properties` tương ứng trong mã nguồn, và — quan trọng nhất với vận hành — **hệ quả khi để trống** từng biến.

Một nguyên tắc xuyên suốt cần nhớ trước: dự án được thiết kế để **xuống thang có kiểm soát** (graceful degradation). Không có biến nào là bắt buộc để khởi động được backend. Thiếu khoá của một dịch vụ thì đúng phần dịch vụ đó tự tắt hoặc trả về rỗng, phần còn lại của API vẫn chạy bình thường. Nhờ vậy chúng ta có thể chạy luồng chính chỉ với JDK và Node.js, và bật thêm tính năng khi có khoá.

Hai lớp biến môi trường

Điểm dễ nhầm nhất khi vận hành là dự án có **hai lớp** biến môi trường, tên không phải lúc nào cũng trùng nhau:

1. Biến trong tệp `.env` — do `docker compose` đọc để thay vào các placeholder

`${...}` trong `docker-compose.yml` / `docker-compose.prod.yml`. Đây là các biến ta sửa khi triển khai (ví dụ `PUBLIC_WEB_URL`, `ADMIN_TOKEN`, `ZCLOUD_BASE_URL`).

1. Biến môi trường của container — cái mà Spring Boot thật sự đọc. Compose chuyển các

biến `.env` sang tên này trước khi trao cho backend. Một số đổi tên (ví dụ `.env: PUBLIC_WEB_URL` container: `RAPPHIM_CORS_ORIGINS`).

Spring Boot ánh xạ biến môi trường của container sang thuộc tính cấu hình bằng **relaxed binding**: viết HOA, thay mọi dấu chấm và gạch ngang bằng gạch dưới. Nhờ đó thuộc tính `rapphim.zcloud.base-url` nhận giá trị từ biến `RAPPHIM_ZCLOUD_BASE_URL`, và bất kỳ thuộc tính nào trong `application.yml` cũng có thể ghi đè bằng biến môi trường theo quy tắc này — kể cả các thuộc tính không khai báo sẵn placeholder `${...}`.

Bảng dưới cho thấy toàn bộ đường đi từ `.env` tới thuộc tính Spring cho các biến có đổi tên hoặc đáng lưu ý:

Biến trong	Biến container (Spring đọc)	Thuộc tính	Ghi chú
PUBLIC_WEB_URL	RAPPHIM_CORS_ORIGINS	rapphim.cors.allowed-origins	Compose lấy PUBLIC_WEB_URL làm origin được phép gọi API
ZCLOUD_BASE_URL	RAPPHIM_ZCLOUD_BASE_URL	rapphim.zcloud.base-url	Đổi tên khi vào container
ADMIN_TOKEN	RAPPHIM_ADMIN_TOKEN	rapphim.admin.token	Đổi tên khi vào container
ZCLOUD_API_KEY	ZCLOUD_API_KEY	rapphim.zcloud.api-key	Giữ nguyên tên
TMDB_ACCESS_TOKEN	TMDB_ACCESS_TOKEN	rapphim.tmdb.access-token	Giữ nguyên tên

Khi chạy backend **không qua Docker** (ví dụ `./mvnw spring-boot:run` lúc phát triển), không có lớp compose, nên chúng ta đặt thẳng biến container. Ví dụ CORS lúc đó đọc từ `RAPPHIM_CORS_ORIGINS`, không phải `PUBLIC_WEB_URL`.

Các lớp cấu hình

Mỗi nhóm thuộc tính `rapphim.*` được nạp vào một record `@ConfigurationProperties` trong gói `com.rapphim.warehouse.config`. Nhờ được ánh xạ này giúp tra ngược từ log hoặc từ mã nguồn về đúng biến cần sửa.

Tiền tố thuộc tính	Lớp	Vai trò
<code>rapphim.provider</code>	<code>ProviderProperties</code>	Base URL và timeout của các nguồn phim công khai (KKPhim, NguonC, VSMOV)
<code>rapphim.tmdb</code>	<code>TmdbProperties</code>	Nguồn metadata TheMovieDB để sinh file NFO
<code>rapphim.zcloud</code>	<code>ZCloudProperties</code>	Kho phim riêng trên homelab (FastAPI + S3)
<code>rapphim.admin</code>	<code>AdminProperties</code>	Khoá và cấu hình trang quản trị
<code>rapphim.anilist</code>	<code>AniListProperties</code>	Nguồn metadata anime (GraphQL)
<code>rapphim.jikan</code>	<code>JikanProperties</code>	Nguồn metadata anime dự phòng (MyAnimeList/Jikan)
<code>rapphim.cors</code>	<code>WebConfig</code> (nạp bằng <code>@Value</code>)	Danh sách origin được phép gọi API

Trừ CORS đọc trực tiếp bằng `@Value` vào một mảng `String[]`, tất cả nhóm còn lại là record bất biến với constructor điền sẵn giá trị mặc định — nghĩa là ngay cả khi `application.yml` bị xoá, các lớp này vẫn tự có mặc định an toàn.

Cổng và URL

Cổng là cấu hình của lớp Docker, không phải của Spring. Bên trong mạng Docker backend luôn nghe 8080 và web nghe 3000; các biến dưới đây chỉ đổi **cổng đưa ra ngoài** (host port).

Biến ()	Mặc định	Bắt buộc?	Mô tả
WEB_PORT	7100	Không	Cổng host cho frontend, ánh xạ tới 3000 trong container
BACKEND_PORT	7101	Không	Cổng host cho backend, ánh xạ tới 8080 trong container
PUBLIC_WEB_URL	http://localhost:7100	Không	Địa chỉ web nhìn từ trình duyệt; compose dùng nó làm origin CORS (xem mục CORS)

Trên homelab, 8080 đã có ZCloud dùng nên dự án chọn cặp cổng 7100 / 7101. Muốn đổi cổng khi triển khai, chỉ cần sửa .env:

```
bash
# .env - doi cong dua ra ngoai
WEB_PORT=8090
BACKEND_PORT=8091
```

Riêng cổng nội bộ 8080 của backend khai báo trong `application.yml` và hiếm khi cần đổi:

```
yaml
server:
  port: 8080
```

Nguồn ZCloud (kho phim homelab)

ZCloud là kho phim riêng chạy trong mạng nội bộ (FastAPI + S3). Nó bắt xác thực, và giống mọi khoá khác, **không được đặt khoá trực tiếp trong mã nguồn**. Cấu hình nạp vào `ZCloudProperties` (`rapphim.zcloud`).

Biến (Spring)	Alias trong	Mặc định	Bắt buộc?	Mô tả
RAPPHIM_ZCLOUD_BASE_URL	ZCLOUD_BASE_URL	http://192.168.100.169:8080	Không	Gốc REST API của ZCloud. Trong .env mặc định là <code>http://host.docker.internal:8080</code> để container gọi ngược về máy chủ
ZCLOUD_API_KEY	ZCLOUD_API_KEY	(rỗng)	Không	Khoá gọi thẳng, gửi qua header <code>x-api-key</code> . Được ưu tiên hơn mật khẩu
ZCLOUD_PASSWORD	ZCLOUD_PASSWORD	(rỗng)	Không	Mật khẩu đăng nhập, chỉ dùng khi chưa có khoá
ZCLOUD_PREFIX	ZCLOUD_PREFIX	(rỗng)	Không	Thư mục gốc chứa phim trong kho. Để rỗng nghĩa là quét cả kho

Biến (Spring)	Alias trong	Mặc định	Bắt buộc?	Mô tả
RAPPHIM_ZCLOUD_PRESIGN_EXPIRY	—	6h	Không	Thời hạn của đường dẫn phát trực tiếp (presigned URL)
RAPPHIM_ZCLOUD_PAGE_SIZE	—	1000	Không	Số mục lấy mỗi lần gọi khi duyệt kho

Logic chọn cách xác thực nằm ngay trong record: có khoá thì dùng khoá, không thì thử mật khẩu.

```
java
// ZCloudProperties.java
public boolean isConfigured() {
    return apiKey != null || password != null;
}

// Khoa gọi thang không cần giữ phiên, nên luôn ưu tiên hơn dạng nhập bằng mật khẩu.
public boolean usesApiKey() {
    return apiKey != null;
}
```

Thuộc tính `prefix` được chuẩn hoá để luôn kết thúc bằng `/` (và bỏ dấu `/` ở đầu), nên ta viết `phim` hay `/phim/` đều cho kết quả như nhau.

Hệ quả khi để trống: thiếu cả `ZCLOUD_API_KEY` lẫn `ZCLOUD_PASSWORD` thì `isConfigured()` trả `false`, **nguồn** `homelab` **tự tắt** — các endpoint của nó trả về danh sách rỗng thay vì làm hỏng cả API. Đây là hành vi cố ý, giống cách TMDB xử lý khi thiếu khoá.

TheMovieDB (TMDB)

TMDB là nguồn metadata để sinh file NFO cho Kodi / Jellyfin / Emby. Cấu hình nạp vào `TmdbProperties` (`rapphim.tmdb`).

Biến (Spring)	Mặc định	Bắt buộc?	Mô tả
TMDB_ACCESS_TOKEN	(rỗng)	Không	Token đọc v4, gửi qua header <code>Authorization: Bearer</code> . Được ưu tiên
TMDB_API_KEY	(rỗng)	Không	Khoá API v3, gửi qua query param <code>api_key</code>
RAPPHIM_TMDB_BASE_URL	<code>https://api.themoviedb.org/3</code>	Không	Gốc REST API của TMDB
RAPPHIM_TMDB_IMAGE_BASE_URL	<code>https://image.tmdb.org/t/p</code>	Không	Gốc CDN ảnh, ghép với đường dẫn tương đối
RAPPHIM_TMDB_LANGUAGE	<code>vi-VN</code>	Không	Ngôn ngữ metadata mong muốn

Cách chọn kiểu xác thực và trạng thái cấu hình được quyết định trong record:

```
java
// TmdbProperties.java
public boolean isConfigured() {
    return apiKey != null || accessToken != null;
}

// Ưu tiên token v4 vì TMDB đang khuyến nghị dùng đang này.
public boolean usesBearerToken() {
    return accessToken != null;
}
```

Hệ quả khi để trống: thiếu cả `TMDB_ACCESS_TOKEN` lẫn `TMDB_API_KEY` thì các endpoint TMDB trả về 503 kèm mã `TMDB_NOT_CONFIGURED`. Riêng endpoint xuất NFO vẫn chạy được bằng dữ liệu của chính nguồn phim, chỉ là thiếu phần metadata bổ sung từ TMDB.

Nguồn metadata anime (AniList và Jikan)

Hai nguồn này chỉ cung cấp metadata anime (không phát video) qua API công khai, **không cần khoá**. Chúng chỉ có duy nhất một base URL, nạp vào `AniListProperties` và `JikanProperties`.

Biến (Spring)	Mặc định	Bắt buộc?	Mô tả
<code>RAPPIM_ANILIST_BASE_URL</code>	<code>https://graphql.anilist.co</code>	Không	Endpoint GraphQL của AniList
<code>RAPPIM_JIKAN_BASE_URL</code>	<code>https://api.jikan.moe/v4</code>	Không	Gốc REST API của Jikan v4, dùng làm nguồn dự phòng cho AniList

Ảnh của cả hai nguồn trả về là URL tuyệt đối nên không cần ghép CDN. Thường ta chỉ đổi hai biến này khi cần trở qua proxy nội bộ.

Khoá quản trị (Admin token)

Trang quản trị cho phép thêm / sửa nguồn phim lúc đang chạy. Cấu hình nạp vào `AdminProperties` (`rappim.admin`).

Biến (Spring)	Alias trong	Mặc định	Bắt buộc?	Mô tả
<code>RAPPIM_ADMIN_TOKEN</code>	<code>ADMIN_TOKEN</code>	(rỗng)	Không	Khoá gọi các endpoint sửa đổi. Nên đặt chuỗi dài, ngẫu nhiên
<code>RAPPIM_SOURCES_FILE</code>	—	<code>data/custom-sources.json</code>	Không	Nơi lưu danh sách nguồn tự thêm. Trong Docker được đặt <code>/app/data/custom-sources.json</code> (gắn với volume)
<code>RAPPIM_ADMIN_CONNECT_TIMEOUT</code>	—	5s	Không	Thời gian chờ mở kết nối tới nguồn tự thêm
<code>RAPPIM_ADMIN_READ_TIMEOUT</code>	—	20s	Không	Thời gian chờ dữ liệu từ nguồn tự thêm

So khoá gửi lên với khoá đã cấu hình được thực hiện ngay trong record:

```
java
// AdminProperties.java
public boolean isConfigured() {
    return token != null;
}

public boolean matches(String candidate) {
    return token != null && token.equals(candidate);
}
```

Hệ quả khi để trống: thiếu `RAPPHIM_ADMIN_TOKEN` thì các endpoint sửa đổi trả về 503 và **trang quản trị chỉ còn xem được**. Đây là mặc định an toàn: thả khoá phần sửa đổi còn hơn để mở cho bất kỳ ai cũng đổi được nguồn phim của cả hệ thống.

Lưu ý về `RAPPHIM_SOURCES_FILE`: trong Docker giá trị này trở vào `/app/data` được gắn với volume `rapphim-data`, nên nguồn tự thêm sống qua các lần dựng lại container. Đổi sang đường dẫn ngoài volume đồng nghĩa mất danh sách nguồn mỗi lần dựng lại ảnh.

Nguồn phim: base URL và timeout

Nhóm này cấu hình các nguồn phim công khai và ngưỡng thời gian chờ khi gọi ra ngoài, nạp vào `ProviderProperties` (`rapphim.provider`). Các thuộc tính này **không khai báo sẵn placeholder** `${...}` trong `application.yml`, nhưng vẫn ghi đề được bằng biến môi trường theo quy tắc relaxed binding.

Biến (Spring)	Mặc định	Bắt buộc?	Mô tả
<code>RAPPHIM_PROVIDER_CONNECT_TIMEOUT</code>	5s	Không	Thời gian tối đa để mở kết nối tới nguồn
<code>RAPPHIM_PROVIDER_READ_TIMEOUT</code>	20s	Không	Thời gian tối đa chờ một phản hồi
<code>RAPPHIM_PROVIDER_KKPHIM_BASE_URL</code>	<code>https://phimapi.com</code>	Không	Gốc REST API của KKPhim
<code>RAPPHIM_PROVIDER_KKPHIM_CDN_IMAGE</code>	<code>https://phimimg.com</code>	Không	Gốc CDN ảnh của KKPhim (ghép với đường dẫn ảnh tương đối)
<code>RAPPHIM_PROVIDER_NGUONC_BASE_URL</code>	<code>https://phim.nguonc.com</code>	Không	Gốc REST API của NguonC
<code>RAPPHIM_PROVIDER_NGUONC_CDN_IMAGE</code>	(rỗng)	Không	NguonC trả URL ảnh tuyệt đối nên gốc CDN để trống
<code>RAPPHIM_PROVIDER_VSMOV_BASE_URL</code>	<code>https://vsmov.com</code>	Không	Gốc REST API của VSMOV
<code>RAPPHIM_PROVIDER_VSMOV_CDN_IMAGE</code>	(rỗng)	Không	VSMOV trả URL ảnh tuyệt đối nên gốc CDN để trống

Các mặc định này được điền ngay trong constructor của record, nên đúng ngay cả khi `application.yml` bị lược bỏ:

java

```
// ProviderProperties.java
public ProviderProperties {
    kkphim = kkphim == null ? new Endpoint("https://phimapi.com", "https://phimimg.com") :
    kkphim;
    nguonc = nguonc == null ? new Endpoint("https://phim.nguonc.com", "") : nguonc;
    // Anh của VSMOV là URL tuyệt đối nên gốc CDN để trong.
    vsmov = vsmov == null ? new Endpoint("https://vsmov.com", "") : vsmov;
    connectTimeout = connectTimeout == null ? Duration.ofSeconds(5) : connectTimeout;
    readTimeout = readTimeout == null ? Duration.ofSeconds(20) : readTimeout;
}
```

Timeout nhận cú pháp `Duration` của Spring: `5s`, `20s`, `1m`, `500ms`... Muốn nói thời gian chờ cho một mạng chậm, ta đặt:

bash

```
# Cho phép chờ lâu hơn khi nguồn phim phản hồi chậm
RAPPHIM_PROVIDER_READ_TIMEOUT=40s
```

CORS

CORS quyết định origin (trình duyệt) nào được gọi thẳng vào API. Giá trị đọc bằng `@Value` vào một mảng `String[]` trong `WebConfig`, tách theo dấu phẩy.

Biến (Spring)	Mặc định	Bắt buộc?	Mô tả
RAPPHIM_CORS_ORIGINS	<code>http://localhost:3000,http://127.0.0.1:3000</code>	Không	Danh sách origin được phép, ngăn cách bằng dấu phẩy

Trong Docker, biến này **không lấy từ** `.env` **trực tiếp** mà được compose điền từ `PUBLIC_WEB_URL`:

yaml

```
# docker-compose.yml (dịch vụ backend)
environment:
  RAPPHIM_CORS_ORIGINS: ${PUBLIC_WEB_URL:-http://localhost:7100}
```

Chính sách CORS áp cho đường dẫn `/api/**` với các phương thức và ngưỡng cache preflight cố định trong mã:

java

```
// WebConfig.java
registry.addMapping("/api/**")
    .allowedOrigins(allowedOrigins)
    .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
    .allowedHeaders("*")
    .maxAge(3600);
```

Một điểm dễ hiểu lầm: **bản thân trang web không cần biến này**. Mọi lời gọi của frontend Next.js đều đi qua route handler của chính nó (server-to-server), không phải từ trình duyệt. `RAPPHIM_CORS_ORIGINS` chỉ có tác dụng khi có ai đó gọi thẳng API từ một trang khác trong trình duyệt. Vì vậy để tránh không làm hỏng trang web; nó chỉ giới hạn lời gọi chéo origin trực tiếp về đúng danh sách mặc định.

Giới hạn RAM (tùy chọn)

Nhóm cuối cùng thuộc lớp Docker và JVM, không phải thuộc tính Spring. Mặc định trong `docker-compose.prod.yml` hợp cho VPS 2GB; chỉ đặt khi cần siết cho máy nhỏ hơn.

Biến ()	Mặc định	Bắt buộc?	Mô tả
BACKEND_MEM_LIMIT	1g	Không	Trần RAM Docker cấp cho container backend
JAVA_OPTS	-Xmx640m	Không	Tùy chọn JVM; ghim trần heap để tránh JVM đọc RAM cả host
WEB_MEM_LIMIT	512m	Không	Trần RAM Docker cấp cho container web

Trên VPS 1GB, `.env.example` gợi ý cập giá trị chặt hơn:

```
bash
# .env - siết cho VPS 1GB
BACKEND_MEM_LIMIT=640m
JAVA_OPTS=-Xmx384m
WEB_MEM_LIMIT=384m
```

Tổng hợp hệ quả khi để trống

Bảng này gom lại các điểm xuống thang quan trọng nhất để tra nhanh khi vận hành:

Biến để trống	Hệ quả
ZCLOUD_API_KEY và ZCLOUD_PASSWORD	Nguồn <code>homelab</code> tự tắt, trả về danh sách rỗng; phần còn lại của API vẫn chạy
TMDB_ACCESS_TOKEN và TMDB_API_KEY	Endpoint TMDB trả 503 <code>TMDB_NOT_CONFIGURED</code> ; xuất NFO vẫn chạy bằng dữ liệu nguồn phim
RAPPHIM_ADMIN_TOKEN	Endpoint sửa đổi trả 503; trang quản trị chỉ còn xem được
RAPPHIM_CORS_ORIGINS	Dùng mặc định <code>http://localhost:3000,http://127.0.0.1:3000</code> ; không ảnh hưởng trang web (gọi server-to-server)
Các base URL / timeout của provider	Dùng giá trị mặc định điền sẵn trong record <code>*Properties</code>

Kết luận

Cấu hình của RapPhim Warehouse xoay quanh vài nguyên tắc gọn: biến `.env` phục vụ lớp Docker, Spring đọc biến container qua relaxed binding, và mọi thứ đều có mặc định an toàn nên thiếu khoá thì đúng phần đó xuống thang chứ không sập cả hệ thống. Khi cần sửa một hành vi, cách làm chuẩn là tra biến trong bảng nhóm tương ứng, đối chiếu với lớp `*Properties` để hiểu mặc định, rồi kiểm tra bảng hệ quả để biết trước điều gì xảy ra khi để trống.

Các tài liệu liên quan trong bộ hướng dẫn:

- 01 – Cài đặt môi trường phát triển — dựng backend

và frontend cục bộ, kèm danh sách biến môi trường tối thiểu để chạy luồng chính.

- Bản xuất tĩnh của API: `docs/openapi.json`, hoặc mở Swagger UI tại `<http://localhost:8080/swagger-ui.html>` khi backend đang chạy.

Hướng dẫn xử lý sự cố

Tài liệu: Hướng dẫn xử lý sự cố RapPhim WareHouse

Phiên bản: 1.0

Cập nhật: 2026-09-11

Đối tượng đọc: DevOps, trực vận hành, lập trình viên

Trạng thái: Chính thức

Tag: sự-cố, chẩn- đoán, vận-hành, backend, mạng, upstream, dns, cors

Giới thiệu

Tài liệu này là sổ tay xử lý sự cố cho RapPhim WareHouse. Nó tồn tại vì một lý do rất cụ thể: hệ thống là một lớp trung gian đứng trước nhiều nguồn bên ngoài (KKPhim, NguonC, VSMOV, kho riêng homelab, TMDB, AniList/Jikan), nên phần lớn sự cố không nằm trong mã của chúng ta mà nằm ở **ranh giới** giữa các tầng — dữ liệu nguồn trả về, logic ứng dụng, và đường mạng ra ngoài. Khi một trang trắng hay một lời gọi API trả 502, câu hỏi đúng không phải "code hỏng chỗ nào" mà là "**tầng nào** đang hỏng".

Trước khi nói *làm thế nào* để sửa từng sự cố, tài liệu đặt ra *cái gì* và *tại sao* của kỷ luật chẩn đoán: **phân tầng trước, quy kết sau**. Bài học đắt nhất của dự án — sự cố 502 UPSTREAM_ERROR ở VSMOV — bắt nguồn từ việc bỏ qua bước này: lỗi bị quy ngay cho Cloudflare chặn TLS của Java, trong khi nguyên nhân thật chỉ là một trường JSON dị dạng. Chúng ta viết lại toàn bộ quy trình để chuyện đó không lặp lại.

Tài liệu gồm ba phần: một bảng tra nhanh "triệu chứng nguyên nhân cách xử lý", một loạt mục chẩn đoán chi tiết cho từng sự cố lớn (kèm lệnh kiểm tra), và cuối cùng là một checklist chẩn đoán nhanh để dán cạnh màn hình trực.

Kỷ luật chẩn đoán phân tầng

Nguyên tắc trung tâm của cả tài liệu: **xác định lỗi ở tầng nào trước khi đổ lỗi cho anti-bot, Cloudflare hay nhà mạng**. Ba tầng, xét theo đúng thứ tự này:

Thứ tự	Tầng	Câu hỏi cần trả lời	Công cụ soi
1	Dữ liệu	Nguồn có trả dữ liệu không? Dữ liệu có đúng hình dạng schema không? Khoá đã cấu hình chưa?	Gọi thẳng nguồn bằng <code>curl</code> , đọc body thô, xem <code>/api/v1/status</code>
2	Ứng dụng	Backend/frontend có xử lý đúng không? Mã lỗi trả về là gì? Cache có cũ không?	Đọc <code>code</code> trong <code>ApiError</code> , log backend, <code>/actuator/health</code>
3	Mạng	Có phân giải được tên miền không? Có mở được kết nối và bắt tay TLS không?	<code>getent hosts</code> , <code>openssl s_client</code> , <code>dig</code>

Chỉ khi cả ba tầng đều "sạch" mà lỗi vẫn còn, và bằng chứng chỉ đúng vào chặn theo tên miền (TCP mở được nhưng TLS bị reset ngay khi gửi SNI), lúc đó mới kết luận là bị chặn ở biên mạng. Quy kết đó là **kết luận cuối cùng, không phải giả thuyết đầu tiên**.

Vì sao thứ tự này quan trọng: quy kết cho "anti-bot" hay "Cloudflare" là cái dễ chịu nhất — nó nghe hợp lý, nó nằm ngoài tầm kiểm soát của ta, và nó cho phép ngừng điều tra. Nhưng nếu nguyên nhân thật ở tầng dữ liệu, mọi công sức xoay quanh TLS/JA3 đều đổ sông. Kỷ luật ở đây gói trong một câu của hiến pháp dự án: **Đo, không đoán**.

Cách hệ thống tự xuống thang

Một điểm thiết kế giúp chẩn đoán dễ hơn: khi một nguồn phụ thiếu khoá hoặc hỏng, hệ thống **xuống thang** thay vì sập cả API. Nhớ rõ hành vi này để đọc đúng triệu chứng:

- Thiếu `ZCLOUD_API_KEY` / `ZCLOUD_PASSWORD` nguồn homelab trả **rỗng**, không ném lỗi.
- Thiếu `TMDB_ACCESS_TOKEN` / `TMDB_API_KEY` nhóm endpoint `/api/v1/tmdb/*` trả

503 `TMDB_NOT_CONFIGURED`; phần còn lại của API, kể cả xuất NFO, vẫn chạy.

- Thiếu `RAPPIM_ADMIN_TOKEN` trang quản trị 503 `ADMIN_NOT_CONFIGURED`, chỉ còn xem được.

Nghĩa là "một tính năng im lặng" thường là cấu hình thiếu ở tầng dữ liệu, **không phải** sự cố mạng. Đừng nhầm xuống thang có chủ đích với hỏng hóc.

Bảng tra nhanh: triệu chứng nguyên nhân cách xử lý

Dùng bảng này để định vị nhanh; mỗi dòng có mục chi tiết tương ứng bên dưới.

Triệu chứng	Tần g nghi ngờ	Nguyên nhân thường gặp	Cách xử lý nhanh
502 với code = <code>UPSTREAM_ERROR</code>	Dữ liệu Mạng	Nguồn trả JSON dị dạng (parse fail) hoặc blip mạng thoáng qua	Gọi thẳng nguồn, đọc body thô; nếu parse fail thì sửa model, nếu chớp nhoáng thì retry đã nuốt
502 với code = <code>UPSTREAM_BLOCKED</code>	Mạng	Không tra được tên miền, kết nối bị cắt ngay khi bắt tay	Kiểm DNS rồi TLS/SNI; đây mới là dấu hiệu chặn theo tên miền
Nguồn homelab trả rỗng	Dữ liệu	Thiếu <code>ZCLOUD_API_KEY</code> / <code>ZCLOUD_PASSWORD</code>	Đặt khoá qua env hoặc trang quản trị; xuống thang là đúng thiết kế
<code>/api/v1/tmdb/*</code> trả 503 <code>TMDB_NOT_CONFIGURED</code>	Dữ liệu	Chưa cấu hình khoá TMDB	Đặt <code>TMDB_ACCESS_TOKEN</code> (v4) hoặc <code>TMDB_API_KEY</code> (v3); kiểm <code>/api/v1/tmdb/status</code>
Trang tải được nhưng mọi lời gọi API bị chặn	Ứng dụng	CORS: origin frontend không nằm trong danh sách cho phép	Đặt <code>RAPPIM_CORS_ORIGINS</code> / <code>PUBLIC_WEB_URL</code> đúng gốc frontend
<code>I/O error</code> , nguồn "hỏng" ngẫu nhiên, đôi khi ra <code>127.0.0.1</code>	Mạng	ISP chèn/đầu đọc phản hồi DNS	Trỏ backend vào CoreDNS DNS-over-TLS (<code>172.29.0.53</code>)
Riêng TMDB không gọi được dù DNS đã đúng	Mạng	Chặn theo tên miền trong bắt tay TLS (SNI)	Cho lưu lượng đi vòng (VPN/tunnel); không sửa được từ mã
Container bị OOM-killer giết, dịch vụ chập chờn	Hạ tầng	Homelab hết RAM (~7.4GB)	Giữ dịch vụ nặng (dub, stack rophim) ở trạng thái tắt; đặt <code>mem_limit</code>
500 với code = <code>INTERNAL_ERROR</code>	Ứng dụng	Lỗi không lường trước phía server	Đọc log backend tại dòng <code>Loi khong mong doi tai ...</code>
400 với code = <code>INVALID_PARAMETER</code> / <code>VALIDATION_FAILED</code>	Ứng dụng	Tham số sai định dạng hoặc ngoài miền (ví dụ <code>limit > 64</code>)	Sửa tham số theo thông điệp lỗi trả kèm

Triệu chứng	Tần g nghi ngờ	Nguyên nhân thường gặp	Cách xử lý nhanh
404 với code = MOVIE_NOT_FOUND	Dữ liệu	Sai slug, hoặc dùng slug của nguồn này cho nguồn khác	Đối chiếu slug đúng nguồn (slug hai nguồn khác nhau)

Bảng mã lỗi tham chiếu

Mọi lỗi API đều trả về envelope `ApiError` với trường `code` để client xử lý theo từng trường hợp. Các mã do `GlobalExceptionHandler` ánh xạ:

	HTT P	Ý nghĩa	Tầng
INVALID_PARAMETER	400	Tham số sai định dạng hoặc ngoài miền giá trị	Ứng dụng
MISSING_PARAMETER	400	Thiếu tham số bắt buộc	Ứng dụng
VALIDATION_FAILED	400	Không qua bước validate (ví dụ <code>limit > 64</code>)	Ứng dụng
UNAUTHORIZED	401	Sai hoặc thiếu khoá quản trị	Ứng dụng
MOVIE_NOT_FOUND	404	Không có phim với slug đã cho	Dữ liệu
ENDPOINT_NOT_FOUND	404	Sai đường dẫn	Ứng dụng
TMDB_NOT_FOUND	404	Phim không có bản ghi tương ứng trên TMDB	Dữ liệu
UPSTREAM_ERROR	502	Nguồn bên ngoài lỗi hoặc timeout, không rõ nguyên nhân	Dữ liệu / Mạng
UPSTREAM_BLOCKED	502	Không tra được tên miền, hoặc kết nối bị cắt khi bắt tay	Mạng
TMDB_NOT_CONFIGURED	503	Chưa cấu hình khoá TheMovieDB	Dữ liệu
ADMIN_NOT_CONFIGURED	503	Chưa đặt RAPPIM_ADMIN_TOKEN	Dữ liệu
INTERNAL_ERROR	500	Lỗi không xác định phía server	Ứng dụng

Hai mã `UPSTREAM_ERROR` và `UPSTREAM_BLOCKED` cùng trả HTTP 502 nhưng mang ý nghĩa chẩn đoán khác hẳn nhau — phân biệt được chúng là chìa khoá của cả tài liệu này. Mã nằm trong `UpstreamException`; hàm `UpstreamException.network(...)` chọn mã dựa trên loại ngoại lệ gốc (`UnknownHostException` và bắt tay TLS bị cắt `UPSTREAM_BLOCKED`; `SocketTimeoutException` và lỗi khác `UPSTREAM_ERROR`).

Kiểm tra sức khoẻ trước khi chẩn đoán

Trước mỗi phiên chẩn đoán, xác nhận bản thân backend còn sống. Đây là bước "tầng ứng dụng" rẻ nhất và loại ngay nửa số giả thuyết.

```
bash
```

```
# Backend con song khong (chay khi backend dang bat)
curl -s http://localhost:8080/actuator/health
```

Kết quả khi ứng dụng khỏe:

```
json
```

```
{"status": "UP"}
```

Xem nguồn nào đang hoạt động và trạng thái cấu hình khoá (không bao giờ lộ giá trị khoá, chỉ báo có/không):

```
bash
```

```
# Cac nguon dang hoat dong
curl -s http://localhost:8080/api/v1/providers

# Backend da co khoa TMDB hay chua
curl -s http://localhost:8080/api/v1/tmdb/status
```

Khi chạy bằng Docker trên homelab, xác nhận cả ba container đều healthy:

```
bash
```

```
docker compose ps
```

Ba container cần thấy là raphim-dns, raphim-backend và raphim-web.

Sự cố 1 — 502 UPSTREAM_ERROR ở VSMOV (bài học phân tầng)

Đây là sự cố kinh điển của dự án và là lý do tồn tại của toàn bộ kỷ luật ở trên.

Triệu chứng. Gọi nguồn vsmov trả 502 với code = UPSTREAM_ERROR. Ban đầu sự cố bị **chẩn đoán sai** là Cloudflare chặn dấu vân tay JA3/TLS của Java — một giả thuyết nghe rất hợp lý vì VSMOV nằm sau Cloudflare.

Nguyên nhân thật. API của VSMOV đôi khi trả `poster_url={}` (một object rỗng) ở chỗ lẽ ra là một chuỗi URL. Jackson gặp object khi mong đợi chuỗi thì **parse fail** — đây là lỗi thuần **tầng dữ liệu**, không liên quan gì tới TLS hay anti-bot. Nếu đã gọi thẳng nguồn và đọc body thô ngay từ đầu, sự cố lộ ra trong một phút.

Bước chẩn đoán

Tầng 1 — dữ liệu. Gọi thẳng nguồn, đọc JSON thô, tìm trường dị dạng:

```
bash
```

```
# Goi thang VSMOV, khong qua backend - xem hinh dang JSON that
curl -s "https://vsmov.com/api/danh-sach/phim-moi-cap-nhat?page=1" | head -c 2000
```

Soi các trường ảnh (`poster_url`, `thumb_url`): nếu thấy `{}` thay vì chuỗi, đó chính là thủ phạm. Đối chiếu qua backend để thấy đúng nguồn nào hỏng:

bash

```
# Gõ qua backend, giới hạn 5 phim để đọc log cho gọn
curl -s "http://localhost:8080/api/v1/movies/latest?provider=vsmov&limit=5"
```

Tầng 2 — ứng dụng. Đọc log backend. Khi `UpstreamException` được ném, handler ghi theo mẫu `Log` từ nguồn `'{provider}': {message}`, còn `VsmovProvider` ghi `Gõ VSMOV thất bại sau {n} lần: ...` sau khi hết lượt retry. Một stack trace của `MismatchedInputException` / `JsonMappingException` xác nhận đây là parse fail, không phải lỗi kết nối.

bash

```
# Log log backend theo từ khóa nguồn (khi chạy bằng Docker)
docker logs raphim-backend 2>&1 | grep -i "vsmov"
```

Tầng 3 — mạng. Chỉ xét tầng này *sau khi* đã loại tầng dữ liệu. Nếu body thô ở tầng 1 trả về bình thường và đầy đủ, mạng không phải vấn đề — đừng nghi Cloudflare tại đây.

Cách khắc phục

Hai thay đổi đã đưa vào mã, phản ánh đúng hai nguyên nhân có thật:

- Chống dữ liệu dị dạng (nguyên nhân chính).** `VsmovProvider` đọc ảnh qua một hàm chỉ nhận giá trị khi nó thực sự là chuỗi, còn `{}` bị coi như ảnh thiếu:

java

```
// VSMOV đôi khi trả {} (object rỗng) cho ảnh thiếu; chỉ nhận khi là chuỗi thật.
private static String asUrl(Object value) {
    return value instanceof String url && !url.isBlank() ? url : null;
}
```

- Nuốt blip mạng thoáng qua (nguyên nhân phụ).** VSMOV nằm sau Cloudflare nên thỉnh

thoảng bị ngắt ngang ngay sau khi container khởi động lại. Lỗi gọi được thử lại tối đa **3 lần** với backoff tăng dần (zooms nhân theo số lần), đủ để vượt qua các cú chớp nhoáng mà không cần đồ cho anti-bot:

java

```
// VSMOV nằm sau Cloudflare nên thỉnh thoảng bị ngắt ngang giữa chừng ("Request cancelled")
// ngay sau khi container khởi động lại. Thu lại vài lần để nuốt các cú chớp nhoáng này.
private static final int NETWORK_ATTEMPTS = 3;
private static final long NETWORK_BACKOFF_MS = 200;
```

Bài học rút ra. Phân tầng trước khi quy kết. Cùng một triệu chứng 502 có thể là parse fail (dữ liệu) hoặc kết nối chậm chờn (mạng), và hai cái đòi hỏi hai cách sửa hoàn toàn khác nhau. Retry không bao giờ sửa được một object JSON dị dạng; sửa model không bao giờ cứu được một kết nối bị cắt.

Sự cố 2 — nguồn homelab trả rỗng

Triệu chứng. Gọi `provider=homelab` trả về danh sách rỗng, trong khi kho thực sự có phim. Không có lỗi, không có 502.

Nguyên nhân. Thiếu cả `ZCLOUD_API_KEY` lẫn `ZCLOUD_PASSWORD`. Đây là hành vi **xuống thang có chủ đích**: nguồn `homelab` trả rỗng thay vì ném lỗi, để phần còn lại của API vẫn chạy bình thường. Rỗng ở đây là dấu hiệu tầng dữ liệu (thiếu cấu hình), không phải tầng mạng.

Bước chẩn đoán

bash

```
# Bien khoa da duoc truyen vao container backend chua
docker exec raphim-backend env | grep -i zcloud
```

Nếu cả hai biến đều trống, đó là nguyên nhân. Lưu ý thêm về mạng nội bộ khi chạy Docker: từ trong container, localhost là chính container, nên ZCloud (chạy trên chính máy chủ, không trong Docker) phải gọi qua `host.docker.internal` — compose đã đặt sẵn địa chỉ này ở `RAPPHIM_ZCLOUD_BASE_URL` và khai báo `extra_hosts: host.docker.internal:host-gateway`.

Cách khắc phục

Đặt một trong hai khoá rồi khởi động lại backend. `ZCLOUD_API_KEY` (gửi qua header `x-api-key`) được ưu tiên; `ZCLOUD_PASSWORD` dùng khi chưa tạo khoá:

bash

```
export ZCLOUD_API_KEY="<khoa goi thang>"
# hoac: export ZCLOUD_PASSWORD="<mat khau dang nhap>"
```

Trên homelab, điền vào tệp `.env` cạnh `docker-compose.yml` rồi dựng lại dịch vụ. Cũng có thể đặt khoá ngay trên trang quản trị `/cms` (khoá đặt qua giao diện được ưu tiên hơn biến môi trường). Để kiểm chứng khoá đã tới nơi: đặt sai khoá thì nguồn trả 502 kèm đúng lời của kho (401 Chưa đăng nhập) — tức là khoá thật sự đi tới ZCloud.

Sự cố 3 — TMDB trả 503 TMDB_NOT_CONFIGURED

Triệu chứng. Nhóm endpoint `/api/v1/tmdb/*` (và trang `/kham-pha`) trả 503 `TMDB_NOT_CONFIGURED`.

Nguyên nhân. Chưa cấu hình khoá TheMovieDB. Đây lại là xuống thang tầng dữ liệu: thiếu khoá thì chỉ nhóm TMDB tắt, còn xuất file NFO vẫn chạy vì KKPhim đã nhúng sẵn mã TMDB/IMDb trong mỗi phim.

Bước chẩn đoán

bash

```
# Backend da co khoa TMDB hay chua (chi bao co/khong, khong lo gia tri)
curl -s http://localhost:8080/api/v1/tmdb/status
```

Cách khắc phục

Đặt token đọc v4 (ưu tiên) hoặc khoá v3, rồi khởi động lại:

bash

```
export TMDB_ACCESS_TOKEN="<read access token v4>"
# hoac khoa v3:
export TMDB_API_KEY="<api key v3>"
cd backend && ./mvnw spring-boot:run
```

Một cái bẫy đã gặp thật: **lạc khoá v3 sang ô v4** (hoặc ngược lại). Nếu đã đặt mà vẫn 503, kiểm tra token nằm đúng biến — token v4 dạng chuỗi dài đặt vào `TMDB_ACCESS_TOKEN`, khoá v3 ngắn hơn đặt vào `TMDB_API_KEY`.

Nếu khoá đã đúng mà TMDB vẫn không gọi được, vấn đề chuyển sang tầng mạng — xem Sự cố 5.

Sự cố 4 — trình duyệt chặn API do CORS

Triệu chứng. Trang tải được phần vỏ nhưng mọi lời gọi API bị chặn; console trình duyệt báo lỗi CORS (thiếu `Access-Control-Allow-Origin`). Bản thân backend vẫn khoẻ khi gọi bằng `curl`.

Nguyên nhân. Đây là lỗi **tầng ứng dụng**, không phải nguồn hay mạng: origin của frontend không nằm trong danh sách CORS được phép của backend. Thường gặp sau khi đổi cổng hoặc địa chỉ triển khai mà quên cập nhật danh sách.

Bước chẩn đoán

Xác nhận backend trả dữ liệu bình thường khi gọi trực tiếp (loại tầng dữ liệu và mạng):

bash

```
# Gọi thẳng backend - nếu ra dữ liệu thì lỗi nằm ở CORS, không phải backend
curl -s "http://localhost:8080/api/v1/movies/latest?limit=1"
```

Kiểm tra header CORS mà backend trả cho đúng origin của frontend:

bash

```
# Giả lập request từ trình duyệt ở gốc frontend
curl -s -I -H "Origin: http://localhost:7100" \
  "http://localhost:8080/api/v1/movies/latest?limit=1"
```

Nếu phản hồi **không** có `Access-Control-Allow-Origin` khớp origin vừa gửi, đó là nguyên nhân.

Cách khắc phục

Đặt danh sách origin cho phép đúng gốc frontend. Backend đọc từ `RAPPHIM_CORS_ORIGINS` (mặc định `http://localhost:3000,http://127.0.0.1:3000`); trên Docker, compose truyền giá trị này vào từ `PUBLIC_WEB_URL`:

bash

```
# Ví dụ cho homelab: frontend chạy ở cổng 7100
export RAPPHIM_CORS_ORIGINS="http://192.168.100.169:7100"
# Trên Docker, đặt PUBLIC_WEB_URL trong .env để compose truyền vào RAPPHIM_CORS_ORIGINS
```

Lưu ý kiến trúc: trong luồng chính, trình duyệt **không** gọi thẳng backend — mọi lời gọi đi qua route handler cùng gốc của Next, nên CORS chỉ ảnh hưởng các lời gọi khác gốc (ví dụ gọi thẳng backend từ mã client). Nếu cả trang bị chặn, kiểm tra thêm `API_INTERNAL_URL` (địa chỉ backend mà máy chủ Next gọi tới) đã đúng chưa.

Sự cố 5 — ISP đầu độc DNS, và chặn TLS theo tên miền

Đây là hai sự cố tầng mạng khác nhau, hay bị gộp làm một. Tách rõ để sửa đúng.

5a. ISP chèn/đầu độc phản hồi DNS

Triệu chứng. Một nguồn "hỏng" ngẫu nhiên với `I/O error`, trông như backend hỏng. Thực chất là **không bao giờ kết nối tới nơi** vì tên miền phân giải ra địa chỉ giả. Đo thật trên máy chủ: hỏi `api.themoviedb.org` năm lần bằng DNS thường, ba lần trả về `127.0.0.1`.

Nguyên nhân. Mạng ở một số nơi chèn phản hồi DNS giả cho vài tên miền. Backend nhận địa chỉ giả rồi gọi vào hư không.

Bước chẩn đoán

bash

```
# Hoi nhieu lan bang DNS thuong - neu ra 127.0.0.1 hoac dia chi doi lien tuc thi la bi dau doc
for i in 1 2 3 4 5; do getent hosts api.themoviedb.org; done
```

Nếu kết quả nhảy loạn hoặc có 127.0.0.1, DNS đang bị đầu độc.

Cách khắc phục

Stack đã có sẵn một container CoreDNS chạy **DNS-over-TLS**: truy vấn đi trong đường mã hoá nên ISP không chen vào được. Backend được trỏ về bộ phân giải này bằng địa chỉ IP tĩnh 172.29.0.53 (ở mức Docker chưa nhận tên dịch vụ, phải ghi IP). Cấu hình forward trong **Corefile**:

```
.:53 {
  forward . tls://8.8.8.8 tls://8.8.4.4 {
    tls_servername dns.google
  }
  cache 300
  errors
}
```

Kiểm chứng qua CoreDNS cho ra địa chỉ thật và ổn định:

bash

```
# Hoi qua bo phan giai rieng - sau lan hoi phai ra cung mot dia chi that
dig @172.29.0.53 api.themoviedb.org +short
```

Nếu container raphim-dns không healthy, backend sẽ rơi lại DNS mặc định và triệu chứng đầu độc quay lại — kiểm tra `docker compose ps` trước.

5b. Chặn TLS theo tên miền (SNI) — không sửa được từ mã

Triệu chứng. Riêng TMDB vẫn không gọi được dù DNS đã đúng. Backend trả 502 với code = UPSTREAM_BLOCKED và thông điệp "Kết nối tới ... bị cắt ngay khi bắt tay".

Nguyên nhân. Cùng mạng đó còn chặn theo tên miền ngay trong bắt tay TLS: TCP mở được tới cổng 443, nhưng vừa gửi tên miền (SNI) là kết nối bị reset. Đây **không phải** chuyện ứng dụng sửa được — và cũng **không phải** anti-bot hay JA3, mà là chặn ở biên mạng.

Bước chẩn đoán

Đây chính là lúc kỷ luật phân tầng cho phép kết luận "bị chặn". Chạy đúng chuỗi đo dưới đây; chỉ khi **tất cả** khớp thì mới quy cho chặn theo tên miền:

bash

```
# 1. TCP toi IP cua dich co mo khong (thay <IP> bang dia chi that)
curl -s -o /dev/null -w "%{http_code}\n" --connect-timeout 5 https://<IP>:443

# 2. Bat tay TLS voi ten mien that - neu bi reset ngay thi la chan theo SNI
openssl s_client -connect <IP>:443 -servername api.themoviedb.org </dev/null

# 3. Doi chung: mot ten mien khac cung ha tang CDN co bat tay duoc khong
openssl s_client -connect <IP>:443 -servername www.google.com </dev/null
```

Kết luận "chặn theo tên miền" chỉ đúng khi: bước 1 mở được, bước 2 bị reset đúng lúc gửi SNI, và bước 3 (tên miền khác, cùng IP/CDN) lại bắt tay bình thường. Thiếu bất kỳ điều kiện nào thì chưa được đổ cho chặn — quay lại tăng dữ liệu và ứng dụng.

`UpstreamException.network(...)` tự nhận diện tình huống này: khi bắt tay bị cắt (`SSLHandshakeException`, `Connection reset, terminated the handshake`), nó gán mã `UPSTREAM_BLOCKED` thay vì `UPSTREAM_ERROR`, để giao diện biết mà hiện hướng dẫn phù hợp.

Cách khắc phục

Không có cách sửa từ trong mã nguồn. Muốn dùng TMDB ở mạng như vậy phải cho lưu lượng đi vòng ra ngoài: VPN, tunnel, hoặc một máy chủ trung gian. Phần còn lại của hệ thống **không phụ thuộc TMDB**: xuất NFO vẫn chạy bằng mã TMDB/IMDb mà KKPhim nhúng sẵn, nên file NFO vẫn đủ `title`, `year`, `plot` và `uniqueid`.

Sự cố 6 — homelab hết RAM, OOM-killer giết dịch vụ

Triệu chứng. Dịch vụ chập chờn, container bị khởi động lại bất ngờ, hoặc `sshd` suýt bị giết. Đây là sự cố **tầng hạ tầng**, không phải nguồn hay ứng dụng.

Nguyên nhân. Homelab chỉ có khoảng 7.4GB RAM. Các khối nặng — nhất là dịch vụ thuyết minh (dub) với ML (PaddleOCR ONNX + ffmpeg + TTS) — vượt quá sức máy, kéo theo OOM-killer ra tay và chém nhầm tiến trình khác.

Bước chẩn đoán

```
bash
```

```
# Bo nho con lai tren may chu
free -h
```

```
# Container nao dang ngon RAM
docker stats --no-stream
```

```
bash
```

```
# Da tung bi OOM-killer ra tay chua (can quyen tren may chu)
dmesg | grep -i "killed process"
```

Cách khắc phục

Giữ các dịch vụ nặng ở trạng thái tắt để giải phóng RAM. Dịch vụ dub **hiện đang tắt** trong `docker-compose.yml`, mã nguồn vẫn còn ở thư mục `dub/`; stack rophim cũng đã được tắt để nhường bộ nhớ. Đặt `mem_limit` cho từng container để khi có OOM thì chỉ một container gánh, không lan sang `sshd` hay dịch vụ khác. Chỉ bật lại dub khi đã có đủ RAM (thêm service, đặt `DUB_INTERNAL_URL` cho web và bật cờ `NEXT_PUBLIC_DUB_ENABLED`).

Kết luận

Sợ chỉ xuyên suốt mọi sự cố trên là một kỷ luật duy nhất: **phân tầng trước, quy kết sau**. Cùng một triệu chứng 502 có thể là JSON dị dạng ở tầng dữ liệu, blip mạng thoáng qua, hay chặn TLS ở biên — và ba cái đòi ba cách sửa khác nhau. Câu chuyện VSMOV nhắc ta rằng "Cloudflare chặn" là giả thuyết dễ chịu nhất và thường là sai nhất khi chưa đo. Đo, không đoán.

Checklist chẩn đoán nhanh

Chạy theo đúng thứ tự này; dừng lại ngay khi tìm ra tầng hỏng.

1. Backend còn sống? `curl -s http://localhost:8080/actuator/health` mong đợi

`{"status": "UP"}`. Nếu không, xem log khởi động backend trước mọi thứ khác.

1. Đọc mã lỗi. Lấy code trong `ApiError`. `TMDB_NOT_CONFIGURED` /

`ADMIN_NOT_CONFIGURED` / nguồn rỗng thiếu cấu hình (tàng dữ liệu), không phải mạng.

1. Tầng dữ liệu — gọi thẳng nguồn. `curl` trực tiếp tới nguồn, đọc body thô. JSON dị

dạng hay trường lạ (`poster_url={}`) lộ ra ở đây. Đây là bước bị bỏ qua trong sự cố VSMOV.

1. Tầng ứng dụng — đọc log. `docker logs raphim-backend | grep -i <nguồn>`. Tìm dòng

Loi tu nguon '...' và loại ngoại lệ (parse fail hay lỗi kết nối).

1. Phân biệt 502. `UPSTREAM_ERROR` = lỗi/parse/timeout (nghi dữ liệu trước);

`UPSTREAM_BLOCKED` = không tra được tên miền hoặc bắt tay bị cắt (nghi mạng).

1. Tầng mạng — DNS. `for i in 1 2 3 4 5; do getent hosts <ten-mien>; done`. Ra

`127.0.0.1` hay địa chỉ nhảy loạn bị đầu độc DNS; kiểm container `raphim-dns` và `dig @172.29.0.53`.

1. Tầng mạng — TLS/SNI. `openssl s_client -connect <IP>:443 -servername <ten-mien>`.

Chỉ kết luận "chặn theo tên miền" khi TCP mở được, SNI bị reset, mà tên miền khác cùng CDN vẫn bắt tay được.

1. Hạ tầng. `free -h` và `docker stats --no-stream`. Container chập chờn kèm RAM cạn

OOM; giữ dịch vụ nặng ở trạng thái tắt.

1. Chỉ bây giờ mới được nói tới anti-bot / Cloudflare / chặn biên — và chỉ khi bằng

chứng ở bước 6–7 đúng vào chặn theo tên miền.

Tài liệu liên quan

- `00-tong-quan-he-thong.md` — kiến trúc hai tầng, danh sách

nguồn, luồng dữ liệu và cache; nền để hiểu tầng nào đứng ở đâu.

- `01-cai-dat-moi-truong-phat-trien.md` — dựng môi

trường, health-check và các biến môi trường tối thiểu.

- `06-tham-chieu-cau-hinh.md` — danh sách đầy đủ

các biến môi trường và cách cấu hình cho từng môi trường.

- `../../README.md` — sổ tay dự án, mục "Giới hạn đã biết" và phần bộ phân giải tên miền riêng.

- `../ebook/van-khan-deploy.md` — coda vui cho sổ tay sự cố,

điểm lại các sự cố deploy có thật dưới dạng "văn khẩn".